University of Mumbai

Re-accredited with A++ Grade (CGPA 3.65) by NAAC (3rd Cycle 2021)

A PROJECT REPORT ON

RPTC Transport Management System & Online Truck Booking Website

"WE DELIVER SAFE." — A Web-Based Transport Management and Booking Platform

SUBMITTED BY

Piyush Sawant

IN PARTIAL FULFILMENT FOR THE AWARD OF

THE DEGREE OF

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

UNDER THE GUIDANCE OF

MRS. SAYALI BORNARE

DEPARTMENT OF COMPUTER SCIENCE

S.I.C.E.S Degree College of Arts, Science & Commerce (Sem VI)

(MARCH: 2025 – 2026)

S.I.C.E.S Degree College of Arts, Science & Commerce,
Jambhul Phata, Chikhloli, Ambernath (W)
NAAC Accredited B+ Grade

Department of Computer Science

CERTIFICATE

This is to certify that Mr. ***Piyush Sawant*** of ***T.Y.B.Sc.CS (Sem VI)*** class has satisfactorily completed the Project ***RPTC Transport Management System & Online Truck Booking Website*** to be submitted in the partial fulfilment for the award of **Bachelor of Science in Computer Science** during the academic year **2025 – 2026**.

Date of Submission: _____

Project Guide

Head of Department of
Computer Science

College Seal

Signature of Examiner

Signature of Principal

D E C L A R A T I O N

I am **Piyush Sawant**, hereby declare that the project entitled ***"RPTC Transport Management System & Online Truck Booking Website"*** submitted in the partial fulfilment for the award of **Bachelor of Science in Computer Science** during the academic year **2025 – 2026** is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the Student: _____

Place: _____

Date: _____

ACKNOWLEDGEMENT

I would want to convey my heartfelt gratitude to **Mrs. Sayali Bornare**, my mentor, for her invaluable advice and assistance in completing my project. She was there to assist me every step of the way, and her motivation is what enabled me to accomplish my task effectively. Her suggestions on the structure of the system, the clarity of the documentation and the discipline of testing helped me convert a simple website into a properly documented software project.

I would also like to thank the **Head of the Department of Computer Science** and all of the other supporting personnel of **S.I.C.E.S Degree College of Arts, Science and Commerce** for enabling me to work on this project by providing me with the necessary skillset, laboratory facilities and academic guidance to accomplish my goals.

I am thankful to **RP TRANSPORT COMPANY (RPTC)**, Kalamboli, Navi Mumbai, for sharing their business details, service list and operating process, which formed the practical basis of the system requirements described in this report. Their real-world booking and dispatch process is what the proposed website is designed to support.

Finally, I would also like to thank my friends and parents for their support and encouragement as I worked on this assignment. Their feedback during the demonstration sessions helped me improve the usability of the website considerably.

Piyush Sawant
T.Y.B.Sc. Computer Science (Sem VI)

CONTENTS

1. Abstract **1**

2. Introduction **2**

2.1 System Development 2

2.2 About the Project — RPTC Transport Management System 2

2.3 Modules of the System 3

2.3.1 Public (Visitor) Module 3

2.3.2 Customer Module 4

2.3.3 Administrator Module 4

3. System Analysis **6**

3.1 Preliminary Investigation 6

3.2 Feasibility Study 6

3.2.1 Technical Feasibility 7

3.2.2 Operational Feasibility 7

3.2.3 Economical Feasibility 7

3.3 Gathering Information 8

3.4 System Study 9

3.4.1 Present (Existing) System 9

3.4.2 Proposed System 9

3.4.3 Limitations of the Proposed System 10

4. System Requirements **11**

4.1 Software Environment 11

4.1.1 Technology Used 11

4.2 Working Environment 14

4.2.1 Hardware Configuration 14

4.2.2 Software Configuration 15

5. System Design **16**

5.1 Introduction to System Design 16

5.2 System Flow Diagram 17

5.3 Data Flow Diagrams 18

5.3.1 Context Level (0 Level) DFD 18

5.3.2 First Level (I Level) DFD 19

5.3.3 Second Level (II Level) DFD — Booking Module 19

5.4 Database Design 21

5.4.1 Data Dictionary 21

5.5 Screens (Project Screen Shots) 27

6. System Testing **41**

6.1 Test Plan 41

6.2 Test Units and Levels of Testing 42

6.3 Other Testing Strategies 43

6.4 Test Cases Executed 43

6.5 Test Deliverables 46

7. Implementation and Evaluation **47**

8. Conclusion and Future Enhancement **50**

8.1 Conclusion 50

8.2 Extensibility 50

8.3 Future Enhancement 50

Bibliography / References **52**

LIST OF FIGURES AND TABLES

List of Figures

Figure 5.1 — System Flow Diagram of RPTC Transport Management System 18
Figure 5.2 — Context Level (0 Level) Data Flow Diagram 19
Figure 5.3 — First Level (I Level) Data Flow Diagram 19
Figure 5.4 — Second Level (II Level) Data Flow Diagram — Booking Module 20
Figure 5.5 — Entity Relationship Diagram of the Data Store 21
Figure 5.6 — Home Page of the Website 27
Figure 5.7 — Quick Truck Booking Form with Simulated Availability Result 28
Figure 5.8 — Services Section of the Home Page 28
Figure 5.9 — “Why Choose Us” Section 29
Figure 5.10 — Four-Step Booking Process Section 29
Figure 5.11 — About Us Page 30
Figure 5.12 — Core Values and Business Details Section 30
Figure 5.13 — Services Page 31
Figure 5.14 — Vehicle Types and Capacities Table 31
Figure 5.15 — Online Truck Booking Form 32
Figure 5.16 — Client-Side Validation Messages 32
Figure 5.17 — Completed Booking Form 33
Figure 5.18 — Booking Request Summary with Generated Reference 33
Figure 5.19 — Freight Quotation Request Form 34
Figure 5.20 — Illustrative Freight Estimate Summary 33
Figure 5.21 — Shipment Tracking Page 35
Figure 5.22 — Tracking Result with Six-Step Progress Timeline 35
Figure 5.23 — “No Record Found” State of Tracking 36
Figure 5.24 — Contact Us Page 36
Figure 5.25 — Validated Contact Form 37
Figure 5.26 — Customer Login and Registration Page 37
Figure 5.27 — Logged-In Demonstration Session 38
Figure 5.28 — Admin Dashboard with Statistics 38
Figure 5.29 — Admin Booking Management Table 39
Figure 5.30 — Vehicle and Fleet Management Panel 39
Figure 5.31 — Responsive Mobile Layout 40

List of Tables

Table 5.1 — LocalStorage Data Store Keys 21
Table 5.2 — Data Dictionary — Booking Record 22
Table 5.3 — Data Dictionary — Freight Quotation Record 23
Table 5.4 — Data Dictionary — Vehicle (Fleet) Record 24
Table 5.5 — Data Dictionary — User and Session Records 24
Table 5.6 — Booking Status Flow Master 25
Table 6.1 — Test Cases and Results 43

1. ABSTRACT

Road transport is the backbone of Indian goods movement, and a large share of it is still coordinated over phone calls, WhatsApp messages and hand-written loading slips. A transport contractor receives booking details in fragments, notes them in a register, informs the driver separately and then answers repeated status calls from the customer. The result is double booking, unclear freight commitments, lost reference numbers and customers who have no reliable way of knowing where their consignment is at any given moment.

The present project, **RPTC Transport Management System & Online Truck Booking Website**, is a web-based front end built for **RP TRANSPORT COMPANY (RPTC)**, a fleet owner and transport contractor based at Kalamboli, Navi Mumbai, that operates on a PAN India basis. The website brings the complete booking conversation onto one platform: the customer selects a route, load details and vehicle type, receives a booking reference number, can request an indicative freight estimate and can track the booking through a six-stage progress timeline. The administrator, in turn, gets a single dashboard where every request can be viewed, approved or rejected, matched with a vehicle from the fleet register, moved through the delivery stages and printed as a booking document.

The system is implemented as a static, framework-free web application using HTML5, CSS3 and vanilla JavaScript. It contains eight linked pages — Home, About Us, Services, Book a Truck, Track Shipment, Contact Us, Customer Login and Admin Dashboard — sharing one stylesheet and one script file. All demonstration records are persisted in the browser's LocalStorage, which makes the complete workflow demonstrable on an ordinary laptop without installing a server or a database. Careful attention has been paid to telling the user what is real and what is illustrative: the availability check, the freight estimate and the admin actions are clearly labelled as demonstration features, and the report describes honestly how each of them must be replaced by a server-side implementation before commercial use.

The present project elucidates the following features:

- Online truck booking request with automatic reference number generation (RPTC-YYYY-NNNNN).
- Route, goods and vehicle selection with complete client-side validation.
- Freight quotation request with an illustrative, clearly-labelled cost estimate.
- Shipment tracking with a six-stage visual progress timeline.
- Customer registration and login (demonstration authentication).
- Admin dashboard for booking approval, vehicle assignment, status updates and fleet records.
- Printable booking summary, click-to-call, WhatsApp enquiry and responsive mobile layout.

2. INTRODUCTION

2.1 System Development

The process of building systems has always been complex; as systems become larger, the costs and complexities get multiplied. So the need for better methods for developing systems is widely recognised to be effective, and the applied model should meet a few basic requirements.

- The model should be structured and cover the entire system development process from feasibility study to programming, testing and implementation.
- The model should utilise established methods and techniques like database design, normalisation and structured programming techniques.
- The model should consist of building blocks, which define tasks, results and interfaces.
- The model should separate the logical system from the physical system.
- Documentation should be a direct result of the development work and should be concise, precise and as non-redundant as possible.

Based on the above requirements of the system model, a system study has been made for this project. Various methodologies have been applied for the system study, evolving design documents, data modelling, input screen design and output (report) design. The development followed the classical waterfall sequence — preliminary investigation, feasibility study, requirement gathering, system analysis, system design, coding, testing and implementation — with short iterations back to design whenever a screen or a validation rule had to be corrected during testing.

2.2 About the Project — RPTC Transport Management System

RP TRANSPORT COMPANY (RPTC) is a fleet owner and transport contractor located at D 5103, Bima Complex, Near Steel Market, Kalamboli, Navi Mumbai – 410218, Maharashtra. The firm arranges full truck load (FTL) and part truck load (PTL) movements, container transportation, open body truck transport, heavy goods movement for industrial and commercial consignments and long-term transport contracting for businesses across India. Its stated promise is the tagline *“WE DELIVER SAFE.”*

Before the proposed system, the booking process of such a transport office is essentially verbal. A customer telephones the office with a route, a material and a tonnage; the person at the desk checks with the fleet in-charge whether a suitable vehicle is free, notes the details in a register, quotes a freight amount verbally and later communicates the driver's mobile number to the customer. Every subsequent status enquiry — *“has the truck left?”*, *“where is the consignment now?”* — is another telephone call. Because nothing is written down in a structured way, the office cannot answer simple questions such as how many bookings are pending confirmation, which vehicle is running on which booking, or what was quoted to a customer last month.

The proposed system moves this conversation onto a website. A visitor can read the company profile and the ten services, check an indicative availability preview, submit a full booking request and receive a reference number of the form **RPTC-2026-10001**, request a freight quotation, and later track the booking through six clearly defined stages. The office gains an admin dashboard in which every request is listed with its route, vehicle type, pickup date and status, and in which the request can be approved, rejected, matched to a vehicle from the fleet register and advanced through the delivery stages, with every change written into a status history.

The project deliberately stops at the front end. It is a complete, working user interface with a browser-side data layer, and it is structured so that a server-side API and a relational database can later be attached behind the same screens without redesigning them. Chapter 7 and Chapter 8 describe exactly which parts are real, which are illustrative and what must be added for production use.

2.3 Modules of the System

The system is divided into three modules: the Public (Visitor) Module, the Customer Module and the Administrator Module. The division follows the three kinds of people who use the website.

2.3.1 Public (Visitor) Module

This module is available to anyone who opens the website and requires no login. It carries the company's identity and the information a prospective customer needs in order to decide whether to book.

- **Home:** The welcome page. It contains the hero banner, the quick truck booking form with its simulated availability preview, four company highlights, ten service cards, a “Why Choose Us” section, a four-step process explanation, clearly-marked sample testimonials and the site footer.
- **About Us:** The company profile — who we are, mission, vision, values, core values grid and the complete business details table (company name, business type, tagline, address, mobile numbers, e-mail, operating area and services).
- **Services:** Detailed descriptions of all ten services with feature lists and booking links, a vehicle type and capacity table (LCV, open body, MCV, container, HCV) and the eight industries served.
- **Book a Truck:** The full six-section booking form (customer, pickup, delivery, goods, truck, payment), the freight quotation form and the generated booking summary.
- **Track Shipment:** Lookup of a booking by reference number with the six-step progress timeline, and a clear “No Record Found” state for an unknown reference.
- **Contact Us:** Office address, click-to-call numbers, WhatsApp button, e-mail link, the validated contact form and an embedded Google Map of the Kalamboli area.

2.3.2 Customer Module

This module belongs to a customer who has registered on the site. Registration stores the name, e-mail address, mobile number and password in the browser's LocalStorage and creates a session record; the login form then restores that session and shows a signed-in panel from which the customer can go straight to booking or tracking, or log out. Registration also prevents the same e-mail address from being registered twice and checks that the two password entries match.

- **Register:** Full name, e-mail, 10-digit mobile number, password and confirm password, with validation on every field.
- **Login:** E-mail and password verification against the stored record.
- **Logged-in panel:** Shows the signed-in customer, with links to *Book a Truck* and *Track Shipment*, and a Logout action that clears the session.
- **My booking activity:** Any booking submitted from the same browser is available for tracking by its reference number.

> **Security note.**
> The customer module is a demonstration. Passwords are stored in plain text in LocalStorage and the session is a local object, so it is not secure and must never be used for real customer accounts. Production requires a secure backend, password hashing (for example bcrypt or Argon2), server-side session management, HTTPS and role-based access control.

2.3.3 Administrator Module

The Administrator Module is the operational heart of the system. It is a single dashboard page that reads the booking, quotation and vehicle records and presents them as live, filterable tables. Statistics cards at the top show total booking requests, pending requests, approved bookings, completed deliveries and quotation requests, all computed from the stored records rather than typed in by hand.

- **Dashboard:** Five statistic cards and a filter for booking status (all, pending, approved, vehicle assigned, picked up, in transit, delivered, rejected).
- **Booking management:** A table of all requests — reference, customer, route, vehicle type, pickup date and status pill — with six actions per row: *view*, *approve*, *reject*, *assign vehicle*, *update status* and *print*.
- **Vehicle and fleet management:** Add, edit and remove vehicles with registration number, type, capacity, driver name, driver contact, availability status, current location and the booking currently assigned.
- **Status history:** Every approval, rejection, vehicle assignment and status change is appended to the booking's history with a timestamp and a note, so the tracking page shows an auditable sequence rather than a single overwritten value.

- **Printing:** Any booking can be printed as a clean booking document; the print stylesheet removes the navigation, banners, buttons and toasts.

Administration note.

In this front end the dashboard is a publicly reachable demonstration page with no login, and all actions are stored only in the local browser. A production deployment must place the dashboard behind backend authentication and role-based authorisation, validate every action on the server and keep an immutable audit log.

3. SYSTEM ANALYSIS

3.1 Preliminary Investigation

First in the system development process is preliminary investigation. Preliminary investigation is conducted in the following phases:

- Project clarification
- Feasibility study
- Project appraisal

Project clarification is the process of selecting a project request for further study. When a system development or modification request is made, the first systems activity — the preliminary investigation — begins. The activity has three parts: request clarification, feasibility study and project appraisal. Many requests from employees and users in an organisation are not clearly stated. Therefore, before any systems investigation can be considered, the project request must be examined to determine precisely what the originator wants. This is called request clarification.

In this project the request came from the transport office itself. The requirement was clarified in discussion with the proprietor and the dispatch staff, and was reduced to four questions that the website must answer for the business:

1. *Can a customer give us the complete load details once, instead of five separate phone calls?* — Answer: a structured booking form with mandatory fields for customer, pickup, delivery, goods, vehicle and payment preference.
2. *Can every enquiry be given a number so that it can be found again?* — Answer: an automatically generated reference in the form RPTC-YYYY-NNNNN and a tracking page that resolves it.
3. *Can the office see, at a glance, what is pending and which vehicle is free?* — Answer: an admin dashboard with status counters, a booking table and a fleet register.
4. *Can the customer follow the consignment without telephoning us?* — Answer: a six-stage progress timeline that is advanced only by an authorised status update.

An important outcome of the preliminary investigation is the determination that the system request is feasible; that determination is recorded in the next section.

3.2 Feasibility Study

The feasibility study is performed to determine whether the proposed system is viable considering the technical, operational and economical factors. After going through the feasibility study we can have a clear-cut view of the system's benefits and drawbacks.

3.2.1 Technical Feasibility

The proposed system is developed using HTML5, CSS3 and JavaScript as the front-end tools, with the browser's LocalStorage as the demonstration data store. No compiler, application server or database server is required to run the system: any modern browser — Chrome, Edge, Firefox or Safari on a desktop, tablet or phone — is sufficient. The only external dependencies are a Google Fonts stylesheet, a Font Awesome icon stylesheet and the Tailwind CSS browser script, all of which are fetched over HTTPS; if they are unavailable the site still functions with system fonts and falls back gracefully.

The development tools used — a plain text editor or Visual Studio Code, the browser's developer tools, and the Lighthouse and console panels for performance and error checking — are free and already installed on the laboratory machines. Because the code is plain HTML, CSS and JavaScript with no build step, the same files can later be served by any web server, including Apache, Nginx, IIS or a Node.js host, and the data layer can be replaced by server calls inside the existing `getData` and `setData` helpers. All the required hardware and software are readily available; hence the system is technically feasible.

3.2.2 Operational Feasibility

The proposed system is operationally feasible because of the following reasons:

- The customer is benefited more as most of his time is saved. A complete booking request can be sent in a few minutes from a phone, without waiting for office hours.
- The transport office no longer transcribes verbal details into a register; the request arrives already structured, with a reference number and a printable summary.
- The interface follows the same steps that the staff already follow on the telephone — customer, pickup, delivery, goods, truck, payment — so no retraining or process change is required.
- The site is fully responsive and works on the low-cost Android phones used by drivers and small traders, not only on office desktops.
- The cost of the proposed system is almost negligible when compared with the benefits gained, because it needs no licence and no server to demonstrate.

3.2.3 Economical Feasibility

As the necessary hardware (an ordinary PC or laptop) and software (a browser and a free code editor) are available at low or no cost, the initial investment is the only cost incurred and the system does not need any further enhancements to remain usable. Hosting of the static site can be done free of charge on any static hosting service, and the only recurring cost for a production version would be a domain name, a small server instance and an e-mail or SMS gateway. Set against the value of avoided double bookings, reduced telephone handling time and better customer retention, the system

is economically feasible; the system is feasible in all respects and hence it encourages taking up the system design.

3.3 Gathering Information

The analysis through collection of data plays the wider role in the analysis of the system. So the data is collected at different levels of management to keep track of full information of the system. The collection of data is done from:

- Top level management — the proprietor of RP TRANSPORT COMPANY, who explained the business model, the service range and the pricing policy;
- Middle level management — the dispatch and fleet in-charge, who explained how vehicles are allotted, what documents accompany a consignment and how status is communicated;
- Low level management — the office assistant who receives calls and maintains the booking register, and drivers who reported what information they actually need on the road.

Different methods were used to collect the data.

Questionnaires. The data is collected through questionnaires by filling a set of questions from the different levels of management. The questions made through questionnaires are of three different types: *structured* questionnaires, in which every question has fixed options (for example, a list of the vehicle types actually arranged by the firm); *unstructured* questionnaires, in which the respondent writes freely about the difficulties of the present process; and *semi-structured* questionnaires, which combine fixed options with a comment column. A short semi-structured questionnaire circulated among five regular customers produced the most useful input — the wish for a reference number and for a status they could check themselves.

Interviews. Interviews were conducted to collect the information, at two levels: formal group interviews with the hierarchical (official) groups in the firm, that is the proprietor with the dispatch staff; and informal group interviews with groups formed outside the company, that is drivers, loading labour and two traders from the Kalamboli steel market.

Observation. The data is also collected by observation of the firm. The office was observed at different timings and in different situations — when the firm is busy, such as the early morning loading window, and when the firm has little work to do — to see how a booking moves from a telephone call to a loaded truck.

Record review. To collect the data and to get a clear idea of the firm, some data was also collected from the past records of the firm: the booking register, loading slips, freight bills and the vehicle list. This information helps very much to get a clear idea of the different problems that occur in different seasons and of some exceptional conditions, such as a vehicle breakdown en route or a consignment

rejected at the delivery point. Reviewing these records is also how the field names of the proposed data store were decided.

3.4 System Study

3.4.1 Present (Existing) System

There are certain features limiting the process of the present system. The drawbacks of the present system are listed below.

- **Everything is verbal.** Bookings, freight amounts and pickup times are agreed on the telephone. Nothing is written down until the loading slip is prepared, so disputes about what was agreed are difficult to settle.
- **No reference number.** A customer who calls again has to describe the load afresh, and the office has to search the register by date and name.
- **No status visibility.** The customer has no way of knowing whether the request was approved, whether a vehicle was assigned, or whether the goods have been picked up; each of these facts requires a telephone call during office hours.
- **Double booking and idle capacity.** Availability is tracked in the fleet in-charge's head or on a whiteboard, so the same vehicle is sometimes committed twice while another stands idle.
- **No management information.** The office cannot state how many requests are pending, how many were rejected, or what proportion of bookings was actually delivered, without counting by hand.
- **No record of the customer.** Repeat customers re-send the same address, GST number and material details for every consignment.
- **Limited working hours.** A trader who decides at night that a load must move next morning cannot register the request until the office opens.
- **Documents are prepared by hand.** The booking confirmation, which the driver must carry, is written manually and is often illegible.

3.4.2 Proposed System

The proposed system, an online transport management and truck booking website, overcomes the drawbacks of the present system. It gives the customer a single place to describe a load, obtain a reference number and follow the consignment, and gives the office a single screen from which every request can be handled. The advantages of the proposed system are listed below.

- The customer can submit a complete, structured booking request at any hour, from a phone or a computer, and immediately receives a reference number of the form RPTC-2026-10001.

- Vehicle availability, capacity ranges and the ten services are published, so the customer chooses the correct vehicle type before speaking to the office.
- A freight quotation request can be raised with route, material, weight and truck type, and the customer gets an illustrative estimate that is explicitly marked as not a valid quote.
- Shipment tracking shows a six-stage timeline — request received, approved, vehicle assigned, picked up, in transit, delivered — so that routine status calls disappear.
- The booking summary can be printed as a clean document for the driver, and the same summary can be forwarded to the office on WhatsApp in one tap.
- The admin dashboard turns the register into live data: counts of pending, approved and completed bookings, a filterable booking table and a fleet register with availability status.
- Every status change is appended to a status history with a timestamp, producing an audit trail for each consignment.
- It is very flexible and user friendly: the layout adapts from a 1440-pixel desktop to a 390-pixel phone, and all forms work with a keyboard alone.
- The person's time and work is reduced very much compared with the present system; the office staff spend their time on dispatch rather than on transcription.
- The customer is not limited to the working hours of the branch: the site is available 24 hours a day, 7 days a week and 365 days of the year.

3.4.3 Limitations of the Proposed System

- The present version is a front-end demonstration. Booking, quotation, vehicle, user and message records are stored only in the browser's LocalStorage, so they are specific to that browser and that device, are not shared between users, and are lost if browser data is cleared. Real multi-user booking requires a backend API and a database.
- The availability check is simulated. It shows an indicative vehicle suggestion for the route and tonnage but does not confirm that a real truck is free.
- The freight figure shown after a quotation request is an illustrative sample calculation, not a valid quotation; RPTC's real freight depends on verified route, vehicle, fuel, toll and loading conditions.
- Tracking reflects only the status recorded by the office. There is no GPS feed, so the position of the vehicle on the road is not shown.
- The login and the admin dashboard have no real security, as described in the notes in Chapter 2.
- The contact form does not deliver e-mail; it stores the message locally for demonstration.

4. SYSTEM REQUIREMENTS

4.1 Software Environment

Software environment is a technical specification of the requirement of a software product. This specifies the environment for development, operation and maintenance of the product. The proposed system is a browser-based application; its environment is therefore described by the protocols, languages and browser interfaces it uses rather than by a database server or an application server.

Technology used:

Layer	Technology used in the project
Transfer protocol	HTTP / HTTPS
Markup and structure	HTML5
Presentation and layout	CSS3 (Flexbox, CSS Grid, media queries, print styles)
Client-side scripting	JavaScript (ES6, vanilla — no framework)
Data store (demonstration)	Web Storage API — *window.localStorage*, JSON serialisation
Browser interfaces used	DOM, DOM events, FormData and the Constraint-free custom validation, IntersectionObserver, URLSearchParams, window.print()
Supporting libraries	Google Fonts (Barlow Condensed, Inter), Font Awesome 6.5.2, Tailwind CSS browser script
Version control	Git and GitHub

4.1.1 HTTP and HTTP Basics

The Hypertext Transfer Protocol is a stateless, TCP/IP-based protocol used for communicating on the World Wide Web. HTTP defines the precise manner in which web clients communicate with web servers. The current versions in use are HTTP/1.1 and HTTP/2, and browsers negotiate HTTPS (HTTP over TLS) for practically every public site. The protocol is standardised in the RFC 7230–7235 series for HTTP/1.1.

The HTTP protocol follows a very simple request/response paradigm. In short, a conversation between a web browser and a web server goes something like this: the client opens a connection to the server, the client makes a request to the server, the server responds to the request, and the connection is closed (or kept alive and reused, in the case of HTTP keep-alive). The four stages of a simple web transaction are:

Stage	What happens in this project
1. Connection	The browser opens a TCP/TLS connection to the host that serves the site files.
2. Request	The browser sends GET /index.html, then further GET requests for *style.css*, *script.js* and the images.
3. Response	The host returns 200 OK with the HTML document, the stylesheet, the script and the image bytes, each with its own MIME type.
4. Close / reuse	The connection is kept alive for the remaining asset requests and released when the page has finished loading.

Because the site is static, every page is an independent HTTP document and navigation between pages is ordinary hyperlink navigation. Form submissions, however, are intercepted in JavaScript with event.preventDefault(), so that the page is not reloaded and the record can be written to LocalStorage and echoed back as a booking summary. This is exactly the place where a production version would instead issue an HTTP POST to a server-side API.

4.1.2 HTML5

HTML5 provides the semantic structure of all eight pages. Each page declares <!DOCTYPE html> and uses semantic elements — *header*, *nav*, *section*, *article*, *footer* — with a viewport meta tag so that the layout is scaled correctly on a mobile phone. HTML5 input types are used where they help the user: *type="date"* opens the device date picker, *type="tel"* opens the numeric keyboard on a phone, *type="email"* triggers e-mail keyboard suggestions, and *type="number"* with *min* and *step* restricts the tonnage field. Attributes such as *required*, *maxlength*, *minlength*, *placeholder*, *autocomplete* and *aria-label* carry the validation contract and the accessibility hints. The document is written to be readable without CSS and is fully usable from the keyboard.

4.1.3 CSS3

A single shared stylesheet of about 760 lines defines the whole visual language of the site. Custom properties (CSS variables) hold the palette and the type scale, so a colour or a radius can be changed in one place:

```
:root{
  --red:#B30000;  --red-dark:#8A0000;  --red-soft:#FDEAEA;
  --navy:#0A2342; --navy-2:#10305C;    --navy-3:#17427E;
  --black:#111111; --ink:#1B2433;      --text:#46536A;
  --muted:#71809A; --grey:#F4F6F8;     --border:#E3E8F0;
  --green:#1E7E34; --amber:#9A6700;
  --radius:14px;   --shadow-md:0 12px 32px rgba(10,35,66,.12);
  --font-head:'Barlow Condensed',sans-serif;
  --font-body:'Inter',sans-serif;
}
```

The layout is built with CSS Grid for the form grids, the service cards, the statistics cards, the process steps and the summary blocks, and with Flexbox for the navigation bar, the top bar and the button rows. Responsive behaviour is controlled by four media queries — 1080 px, 1024 px, 900 px and 600 px — which progressively collapse the grids from five columns to two and then to one, and which replace the horizontal menu with a hamburger menu below 900 px. A dedicated @media print block removes the top bar, navigation, banners, floating buttons and toast container, and converts the coloured cards to black-on-white bordered boxes, so that a booking summary prints as a clean document.

4.1.4 JavaScript

The behaviour of all pages is contained in one script file of about 1000 lines, written as a single immediately-invoked function expression in strict mode so that no variable leaks into the global scope. It is organised in numbered sections: utility helpers and company details, LocalStorage helpers, the booking status model, reference-number generators, seed demonstration data, toast and modal helpers, navigation, scroll-reveal animation, form validation, and one initialiser function per page that returns early when its own markup is absent. The last property means a single script file can be included by every page without errors.

The central data functions are only a few lines long, and they are the single point through which every page reads and writes data. Replacing them with *fetch()* calls against a backend API is the natural upgrade path to a multi-user system:

```
const KEYS = {
  bookings:'rptc_bookings', quotes:'rptc_quotes', vehicles:'rptc_vehicles',
  users:'rptc_users',       messages:'rptc_messages', session:'rptc_session'
};
const getData = (key, fallback) => {
  try { const v = JSON.parse(localStorage.getItem(key));
        return v == null ? fallback : v; }
  catch (e) { return fallback; }
};
const setData = (key, value) => localStorage.setItem(key, JSON.stringify(value));
```

The script also implements the features that make the interface feel like an application: a toast notification that appears for three seconds and slides away, a reusable modal dialog used by the admin actions, an IntersectionObserver that reveals sections as they scroll into view, a sticky navigation bar that shrinks on scroll, a back-to-top button that appears after 500 px, automatic pre-filling of the booking form from the query string of the home page quick-booking form, and automatic injection of today's date as the minimum value of every date field so that a past pickup date cannot be chosen.

4.1.5 Web Storage (LocalStorage) as the Data Store

The Web Storage API gives a web page a simple key/value store, roughly 5 MB per origin, that survives a page reload and a browser restart. Records are stored as JSON strings under the six keys listed above. The API is used because it allows the entire workflow — submit a booking, see it in the admin table, approve it, assign a vehicle, advance its status, and watch the tracking timeline change — to be demonstrated on a single laptop with no installation whatsoever.

LocalStorage is synchronous and origin-scoped. Its limitations are important and are stated openly in the system itself: it is per-browser and per-device, it is not shared between users, it is not backed up, it can be cleared by the user, and it offers no access control. The data model has therefore been designed to be exactly the model that a relational database would use (Chapter 5.4), so that the storage layer can be swapped without touching the screens.

4.1.6 Supporting Libraries

- **Google Fonts** — supplies *Barlow Condensed* for headings and *Inter* for body text. Two families and five weights only, to keep the page light.
- **Font Awesome 6.5.2** — supplies the truck, container, shield, phone, envelope, WhatsApp and chevron icons used throughout the interface, so that no raster icon files are needed.
- **Tailwind CSS browser script** — loaded for a handful of utility classes used in the demonstration banners; the substantive styling remains in the hand-written stylesheet, so the site continues to look correct if the script is blocked.

4.1.7 Version Control

The complete source — eight HTML pages, the stylesheet, the script file and a README describing how to run and what is real versus demonstration — is maintained in a Git repository published on GitHub. Version control made it possible to try changes to the layout and the validation rules without losing a working version, and it allows the project guide to run exactly the same code that was demonstrated.

4.2 Working Environment

The working environment describes the hardware and software configuration on which the system was developed and on which it is expected to run.

4.2.1 Hardware Configuration

Component	Minimum requirement
Processor	Intel Core i3 (2.0 GHz) or equivalent; Core i5 recommended
RAM	4 GB minimum; 8 GB recommended for editing and testing together
Hard disk drive	20 GB free space (the project folder itself is under 1 MB)
Keyboard	104 keys, standard QWERTY
Mouse	Standard optical mouse or touchpad
Monitor	15.6″ colour display, 1366 × 768 or higher
Display type	HD / LED; a full-HD display is preferred for design work
Mobile device (for responsive testing)	Android phone, 390 px wide viewport, or the browser device-emulation mode
Internet connection	Required only for the web-font and icon CDNs, the WhatsApp links and the embedded map

4.2.2 Software Configuration

Component	Configuration used
Operating system	Windows 10 / 11, Linux (Ubuntu) or macOS
Web browser	Google Chrome 100+ / Microsoft Edge 100+ / Mozilla Firefox 100+ / Safari 15+
Web server	Not required — pages open directly from the file system. Optional static server: `python -m http.server 8000`
Designing tool	Visual Studio Code / Notepad++ with HTML, CSS and JavaScript syntax support
Client-side scripting	JavaScript (ES6), enabled in the browser
Data store	Browser LocalStorage (no database server required for the demonstration)
Debugging tools	Browser developer tools — Console, Elements, Network and Lighthouse panels
Version control	Git 2.x with a GitHub remote
Screen capture / documentation	Browser screenshot and print-to-PDF for this report

How to run the project.

Copy the whole project folder to the computer, keeping all files together; then double-click *index.html*. No installation, build step or server is required. All navigation between pages works through the menu, the buttons and the footer. For a phone test on the same Wi-Fi network, serve the folder with any local static server and open the computer's address from the phone.

5. SYSTEM DESIGN

5.1 Introduction to System Design

System design is the process of planning a new system or of replacing the existing system. Simply stated, system design is like the blueprint for a building: it specifies all the features that are to be in the finished product. The system design phase follows the system analysis phase. Design is concerned with identifying functions, the data streams among those functions, maintaining a record of the design decisions and providing a blueprint for the implementation phase.

Design is the bridge between system analysis and system implementation. Some of the essential fundamental concepts involved in the design of application software are:

- **Abstraction**
- **Modularity**
- **Verification**

Abstraction is used to construct solutions to a problem without having to take account of the intricate details of the various component sub-problems. Abstraction allows the system designer to make step-wise refinement, in which each stage of the design may hide unnecessary details associated with representation or implementation from the surrounding environment. In this project the abstraction is visible in three levels: the page level (seven public screens and one admin screen), the section level (the six numbered sections of the booking form) and the data level (the six LocalStorage keys accessed only through *getData()* and *setData()*, so that no page knows how the data is stored).

Modularity is concerned with decomposing the main module into well-defined, manageable units with well-defined interfaces among the units. This enhances design clarity, which in turn eases implementation, debugging, testing, documenting and maintenance of the software product. Modularity viewed in this sense is a vital tool in the construction of large software projects. Here each page is a module with its own initialiser function, the behaviour is separated from the presentation (one script file, one stylesheet, eight documents), and the demonstration data is seeded only once by a single function.

Verification is a fundamental concept in software design. A design is verifiable if it can be demonstrated that the design will result in an implementation that satisfies the customer's requirements. Verification is of two types, namely:

- verification that the software requirements analysis satisfies the customer's needs;
- verification that the design satisfies the requirement analysis.

Both were addressed in this project through a requirements checklist agreed with the firm, a traceable mapping from each requirement to a screen (Chapter 5.5) and a formal set of test cases executed against the finished screens (Chapter 6.4).

5.2 System Flow Diagram

The system flow diagram shows the path followed by one booking, from the moment the customer opens the website to the moment the consignment is delivered and the tracking timeline shows all six stages completed. It also shows the two feedback loops in the design: the validation loop, which returns an invalid form to the customer with a specific message, and the administration loop, through which the office controls the status that the customer sees.

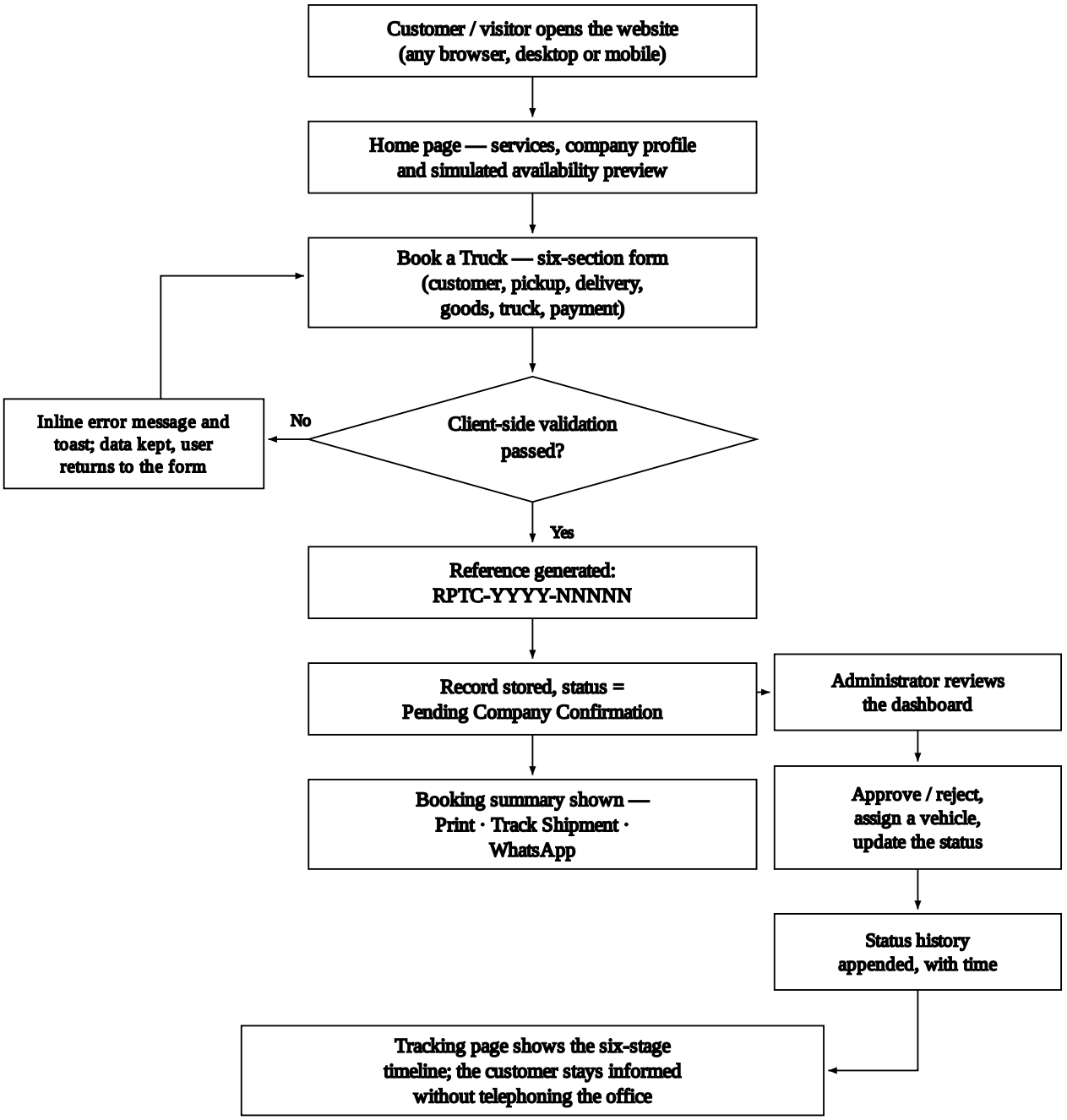
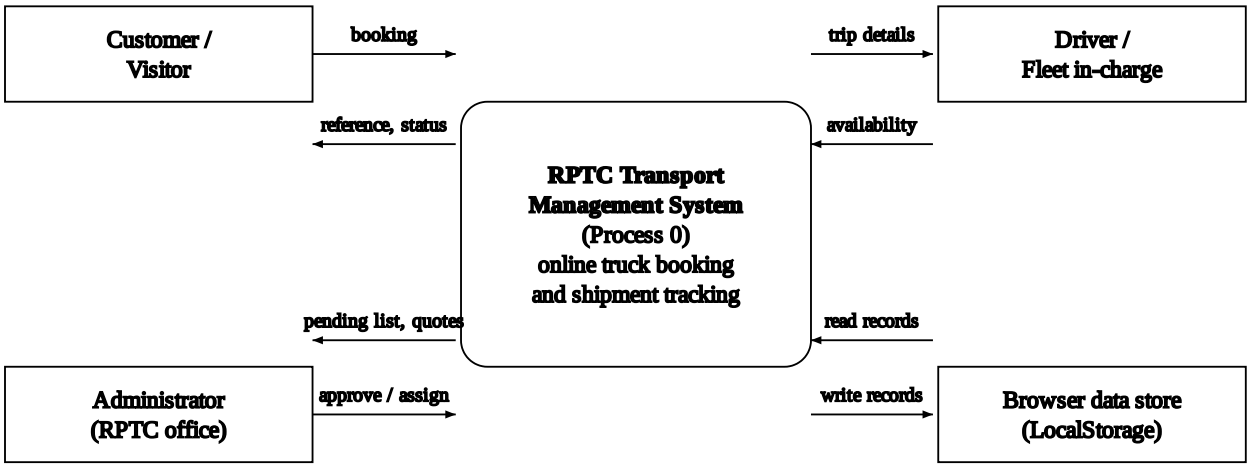


Figure 5.1 System Flow Diagram of the RPTC Transport Management System

5.3 Data Flow Diagrams

The data flow diagrams describe the same system as a set of processes, external entities and data stores, and show what data enters and leaves each process. Three levels are presented: the context (0) level, which treats the whole website as one process; the first level, which decomposes it into six major processes; and the second level, which decomposes the booking process into its seven internal steps. Notation used: a **rectangle** is an external entity, a **rounded rectangle** is a process, an **open-ended rectangle** is a data store and a **labelled arrow** is a data flow.

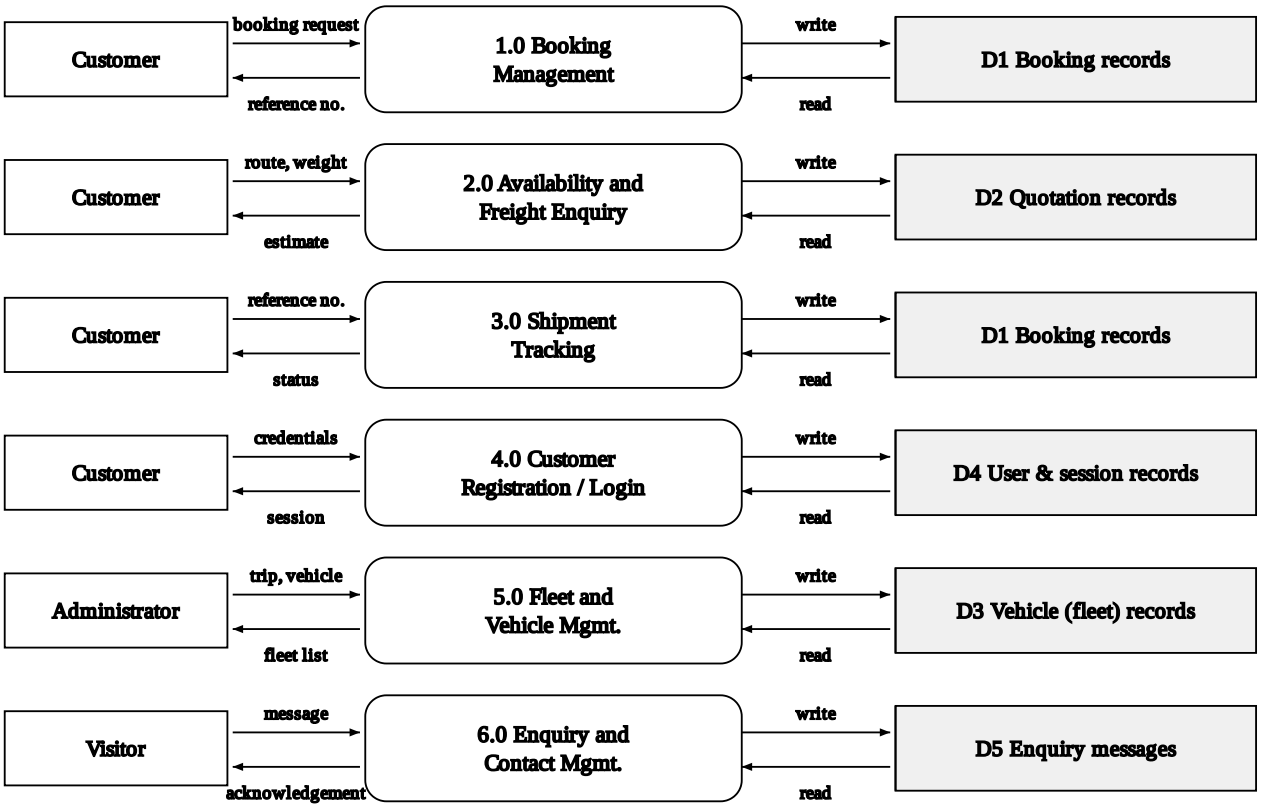
5.3.1 Context Level (0 Level) DFD



Validation rules, the status model and reference numbering are applied inside Process 0.

Figure 5.2 Context Level (0 Level) Data Flow Diagram

5.3.2 First Level (I Level) DFD

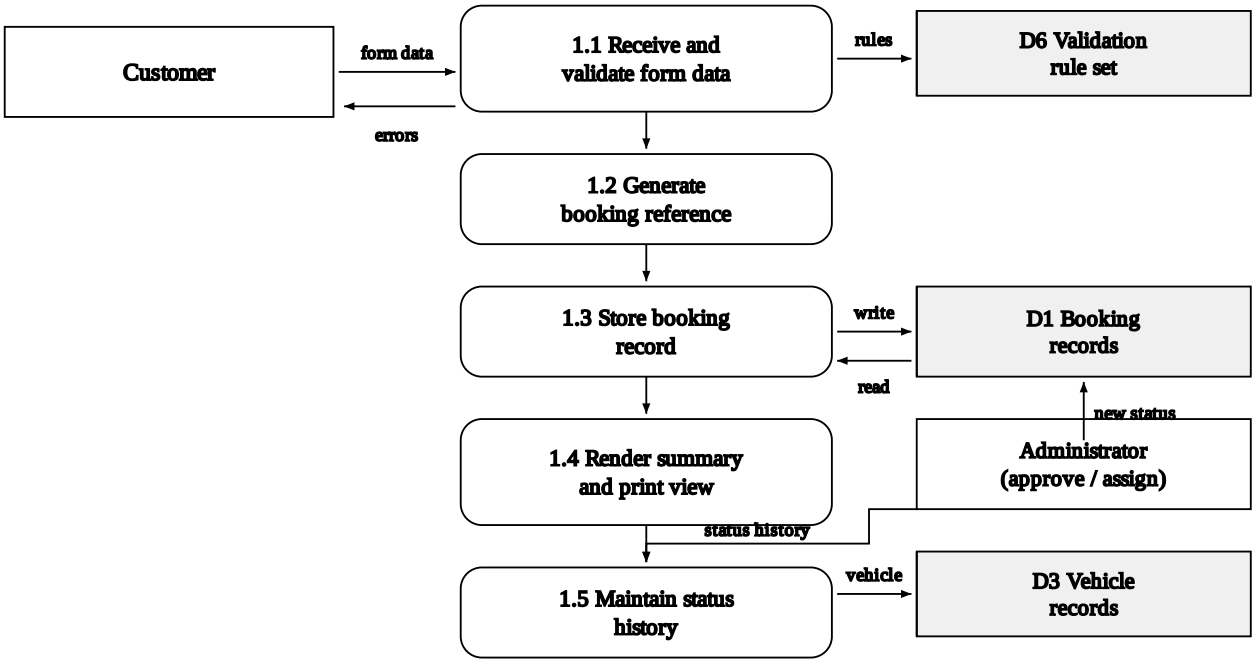


Processes 1.0–6.0 correspond to the initialiser functions of the eight pages of the website.

Figure 5.3 First Level (I Level) Data Flow Diagram

5.3.3 Second Level (II Level) DFD — Booking Module

The second level diagram expands process 1.0 (Booking Management) into the seven steps that a booking actually passes through inside the website, and shows the interaction between the customer, the validation rules, the record store and the administrator.



The administrator loop is the only path by which a booking status or an assigned vehicle can change.

Figure 5.4 Second Level (II Level) Data Flow Diagram — Booking Module

5.4 Database Design

A database is an integrated collection of data and provides a centralised access to the data and makes it possible to treat data as a separate resource. Usually centralised data-managing software is called a Relational Database Management System (RDBMS). The most significant difference between an RDBMS and other types of data management is the separation of data as seen by the program and data as stored on the direct access storage device. This is the difference between logical and physical data.

The present implementation keeps its records in the browser's LocalStorage, which is a key/value store rather than a relational database; however, the records themselves follow a relational shape. Each key holds an array of objects, every object has a fixed primary identifier (*ref* for a booking, *ref* for a quotation, *id* for a vehicle, *email* for a user) and the nested groups inside a booking (customer, pickup, delivery, goods, vehicle, payment, history) correspond exactly to the tables into which the data would be normalised when the system is moved to a server database. Table 5.1 lists the keys; Tables 5.2 to 5.6 describe the logical tables; Figure 5.5 shows the corresponding entity relationship diagram.

Table 5.1 — LocalStorage Data Store Keys

Storage key	Logical table	Contents
rptc_bookings	Booking	Every booking request with its customer, route, goods, vehicle requirement, payment preference and status history.
rptc_quotes	Freight quotation	Every quotation request with route, material, weight, truck type, contact number and the illustrative estimate.
rptc_vehicles	Vehicle / fleet	The fleet register — registration number, type, capacity, driver, contact, availability, location and assigned booking.
rptc_users	Customer account	Registered customers — name, e-mail (unique), mobile number and password.
rptc_messages	Enquiry / message	Contact-form submissions — name, mobile, e-mail, subject and message.
rptc_session	Session	The signed-in customer's name, e-mail and login time, removed on logout.

5.4.1 Data Dictionary

The efficiency of an application developed using an RDBMS mainly depends upon the database tables, the fields in each table and the way the tables are opened using the contents in them to retrieve the necessary information. Hence a careful selection of tables and their fields is imperative. The

tables used in this system are created keeping the above points in mind. The tables used are given below; *PK* marks the primary key and *FK* a foreign key.

Table 5.2 — Data Dictionary: Booking Record (*rptc_bookings*)

Field	Data type	Size / rule	Description
ref	Text (PK)	15	Booking reference generated as RPTC-YYYY-NNNNN, e.g. RPTC-2026-10001. Unique for every request.
createdAt	Date / time	ISO 8601	Date and time at which the request was submitted; used for sorting and for the “last updated” display.
customer.name	Text	60	Full name of the person booking the truck. Mandatory.
customer.company	Text	80	Firm or company name, if the booking is on behalf of a business. Optional.
customer.mobile	Text	10	Indian mobile number, validated against */^[6-9]\d{9}$/*. Mandatory.
customer.email	Text	80	E-mail address, validated for the presence of “@” and a domain. Optional but validated when supplied.
customer.gst	Text	15	GSTIN of the consignor, used on the freight bill. Optional.
pickup.address, city, state, pin	Text	120 / 40 / 40 / 6	Where the goods are to be collected. PIN is validated as exactly six digits.
pickup.date, time	Date / text	—	Requested pickup date (cannot be in the past) and preferred pickup window.
delivery.address, city, state, pin	Text	120 / 40 / 40 / 6	Where the goods are to be delivered.
goods.material	Text	80	Name of the material being moved. Mandatory.
goods.category	Text	30	Material category chosen from a fixed list (perishable, fragile, general, etc.).
goods.weight	Decimal	> 0	Approximate consignment weight in tonnes.
goods.packages	Number	—	Number of packages, bags or cartons.
goods.dimensions	Text	60	Optional dimensions of the largest package or of the loaded lot.
goods.handling	Text	200	Special handling instructions, e.g. “keep covered, do not stack”.

Field	Data type	Size / rule	Description
vehicle.type	Text	40	Requested vehicle category — open body, container, LCV, MCV or HCV.
vehicle.capacity	Text	20	Required capacity band, e.g. “Up to 9 T”.
payment.freight	Text	20	Freight payment preference — sender pays, receiver pays or to be paid.
payment.advance	Text	5	Whether an advance is offered (Yes / No).
payment.instructions	Text	200	Any billing or delivery instruction from the customer.
status	Text	30	Current stage of the booking; one of the seven values in Table 5.6.
vehicleReg	Text (FK)	20	Registration number of the vehicle assigned by the administrator; empty until assignment.
history[]	Array	—	Status history — one entry per change, each holding status, timestamp and note.
sample	Boolean	—	True for the seeded demonstration records, so that the interface can label them “Sample data”.

Table 5.3 — Data Dictionary: Freight Quotation Record (*rptc_quotes*)

Field	Data type	Size / rule	Description
ref	Text (PK)	17	Quotation reference in the form RPTC-Q-YYYY-NNNN.
createdAt	Date / time	ISO 8601	Date and time of the quotation request.
pickupCity, deliveryCity	Text	40 each	Origin and destination of the proposed movement.
material	Text	60	Type of material to be moved.
weight	Decimal	> 0	Weight in tonnes; used in the illustrative calculation.
truck	Text	40	Requested truck type; selects the base rate in the illustrative calculation.
date	Date	—	Proposed pickup date.
contact	Text	10	Contact number for the quotation.
status	Text	20	“AWAITING QUOTATION” until the office issues the real freight amount.
estimate.low, estimate.high	Number	INR	Illustrative range derived from the base rate plus a per-tonne component, rounded to the nearest 500.

Field	Data type	Size / rule	Description
estimate.illustrative	Boolean	—	Always true in this version; drives the “sample only — not a valid quote” warning.

Table 5.4 — Data Dictionary: Vehicle / Fleet Record (*rptc_vehicles*)

Field	Data type	Size / rule	Description
id	Text (PK)	12	Internal vehicle identifier, V-001 onwards.
reg	Text	20	Registration number as painted on the vehicle, e.g. MH-48-AB-1001.
type	Text	40	Vehicle category — open body, container, LCV, MCV or HCV.
capacity	Text	20	Rated capacity, e.g. “9 T” or “16 T (Multi-axle)”.
driver	Text	60	Name of the driver currently attached to the vehicle.
contact	Text	10	Driver's mobile number, validated as a 10-digit Indian number.
availability	Text	15	Available / On Trip / Maintenance.
location	Text	60	Last known position of the vehicle, e.g. “Kalamboli, Navi Mumbai”.
assigned	Text (FK)	15	Reference of the booking on which the vehicle is currently deployed.

Table 5.5 — Data Dictionary: User and Session Records (*rptc_users, rptc_session, rptc_messages*)

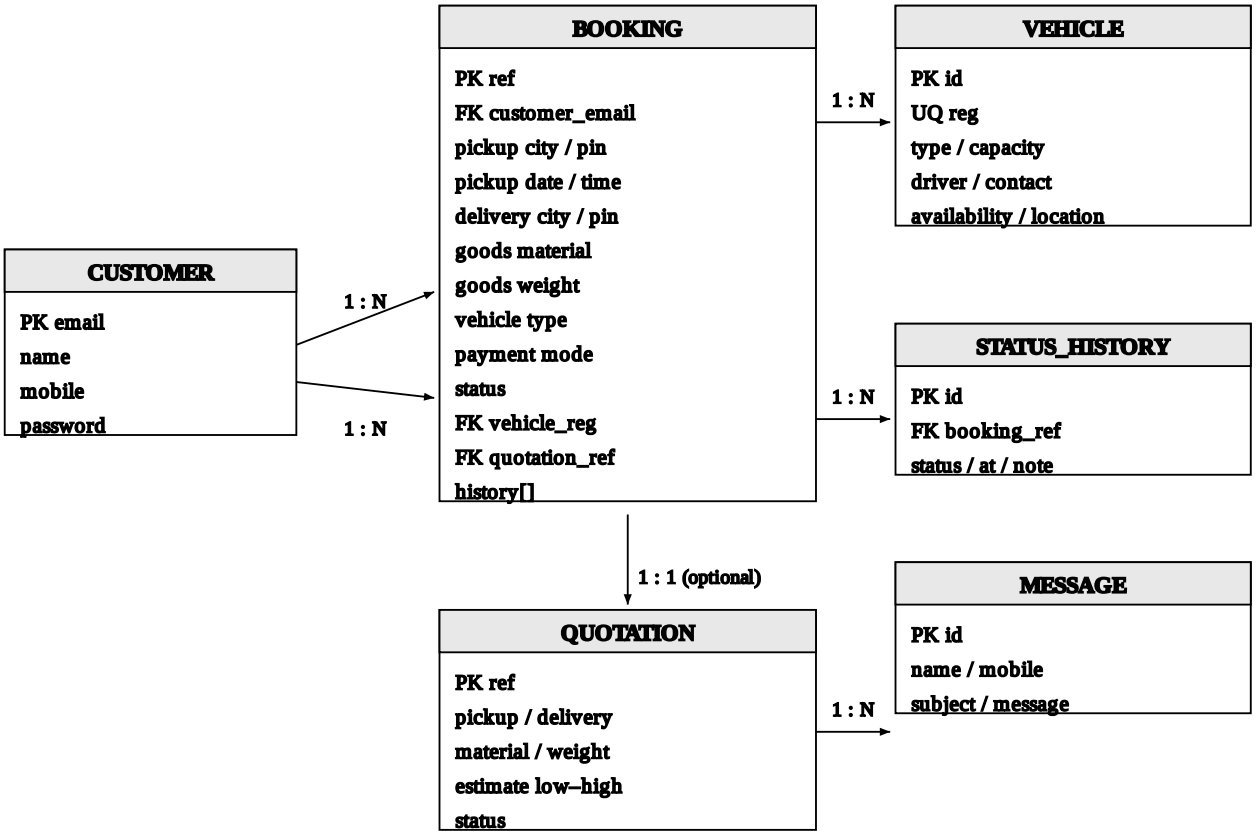
Field	Data type	Size / rule	Description
users.email	Text (PK)	80	Login identity of the customer. Duplicate registration is rejected.
users.name	Text	60	Full name given at registration.
users.mobile	Text	10	10-digit mobile number, validated on registration.
users.password	Text	64	Password, minimum six characters. Stored unencrypted in this demonstration only.
users.createdAt	Date / time	ISO 8601	Date of registration.
session.name, email	Text	60 / 80	Copy of the signed-in customer's identity used to render the logged-in panel.
session.at	Date / time	ISO 8601	Time of login; the record is removed on logout.
messages.name, mobile, email	Text	60 / 10 / 80	Identity of the person sending a contact-form enquiry.

Field	Data type	Size / rule	Description
messages.subject, message	Text	60 / 500	Subject chosen from a fixed list and the free-text message.

Table 5.6 — Booking Status Flow Master

Step	Status key	Label shown to the customer	Set by
1	PENDING COMPANY CONFIRMATION	Booking Request Received	System, on submission of the form
2	APPROVED	Booking Approved	Administrator (approve action)
3	VEHICLE ASSIGNED	Vehicle Assigned	Administrator (assign vehicle action)
4	PICKED UP	Goods Picked Up	Administrator (update status action)
5	IN TRANSIT	In Transit	Administrator (update status action)
6	DELIVERED	Delivered	Administrator (update status action)
—	REJECTED	Booking request rejected	Administrator (reject action)

Entity Relationship Diagram



These cardinalities are the foreign keys a SQL schema should declare.

Figure 5.5 Entity Relationship Diagram of the Data Store

The relationships are: one customer may raise many bookings, while a booking without login is allowed and therefore the customer link is optional; one vehicle may be assigned to many bookings over time but to only one at a time, which is what the *availability* and *assigned* fields enforce in the dashboard; one booking has many status-history entries and exactly one current status; and one quotation may lead to at most one booking. These cardinalities are the ones a future SQL schema should declare as foreign keys.

5.5 Screens (Project Screen Shots)

The screens reproduced in this section are actual screen captures of the completed website, taken at a desktop resolution of 1440 × 860 pixels and, for the last figure, at a mobile resolution of 390 × 844 pixels. The demonstration records visible in the admin and tracking screens are those created while testing: the sample records that the site seeds itself, plus the booking and quotation requests submitted during the test run described in Chapter 6.

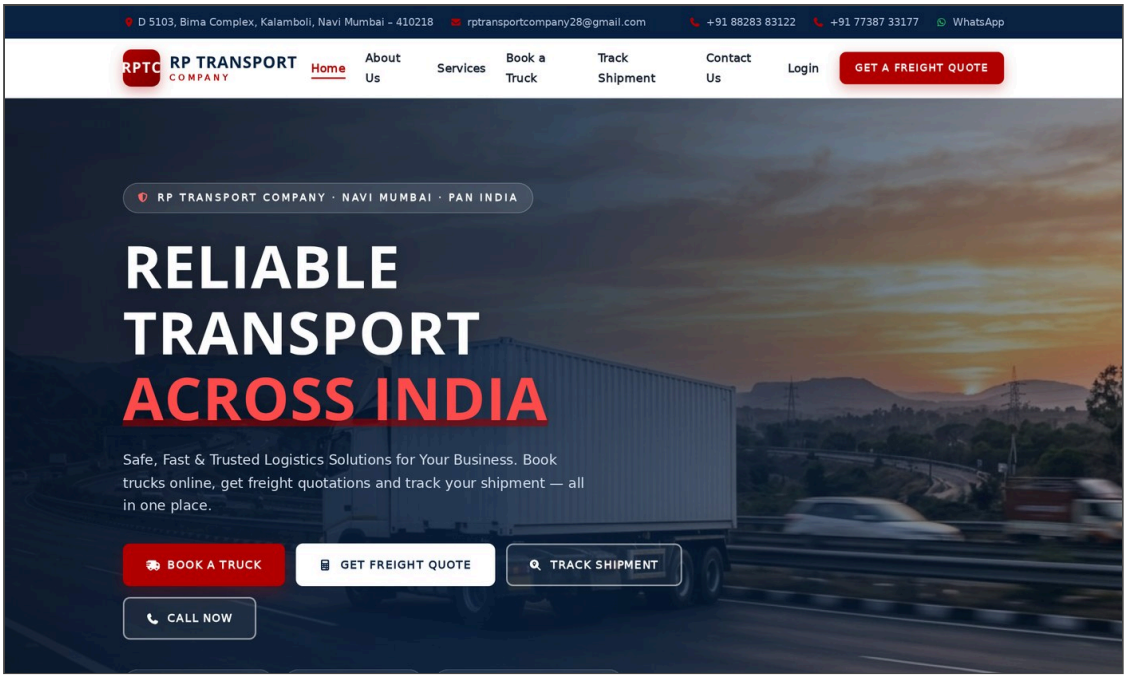


Figure 5.6 — Home page: hero banner, tagline, primary actions and the quick booking form entry point

The home page establishes the identity of the firm in one screen: the company name and the promise *“WE DELIVER SAFE.”*, the fact that it operates from Navi Mumbai on a PAN India basis, and four immediate actions — Book a Truck, Get Freight Quote, Track Shipment and Call Now. The top bar carries the registered address, the e-mail address and both mobile numbers as click-to-call links, so that a visitor on a phone can reach the office in one tap.

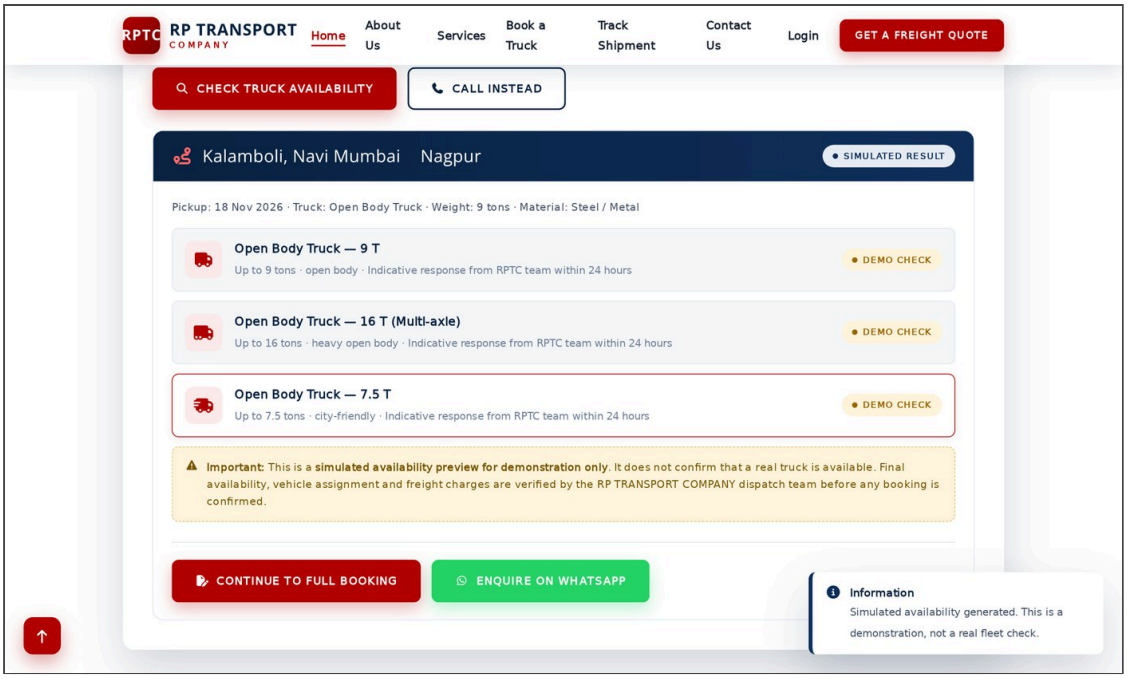


Figure 5.7 — Quick truck booking form and the clearly-labelled simulated availability result

The quick booking form on the home page collects only what is needed to judge feasibility: pickup and delivery city, pickup date, material, truck type, weight, name and mobile number. On submission the page shows an indicative list of vehicle categories for the route and tonnage. The result carries a “Simulated result” pill and an explicit warning that it does not confirm a real truck; the visitor is then offered the full booking form or a WhatsApp enquiry.

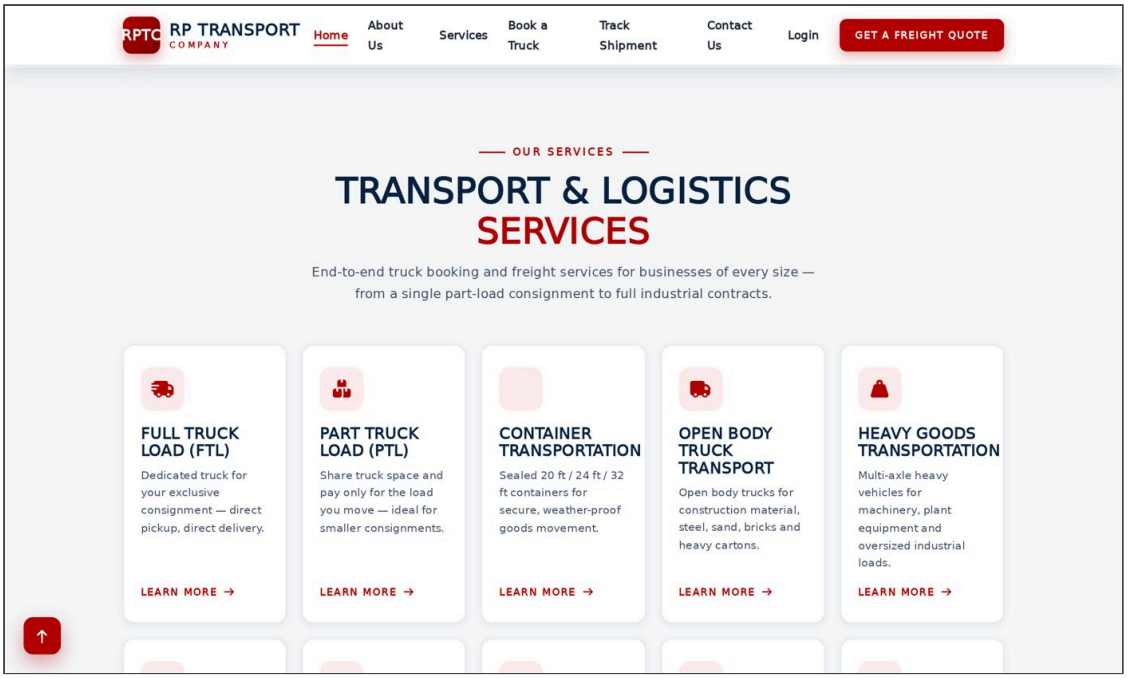


Figure 5.8 — Services section of the home page: ten service cards, each linking to its detailed description

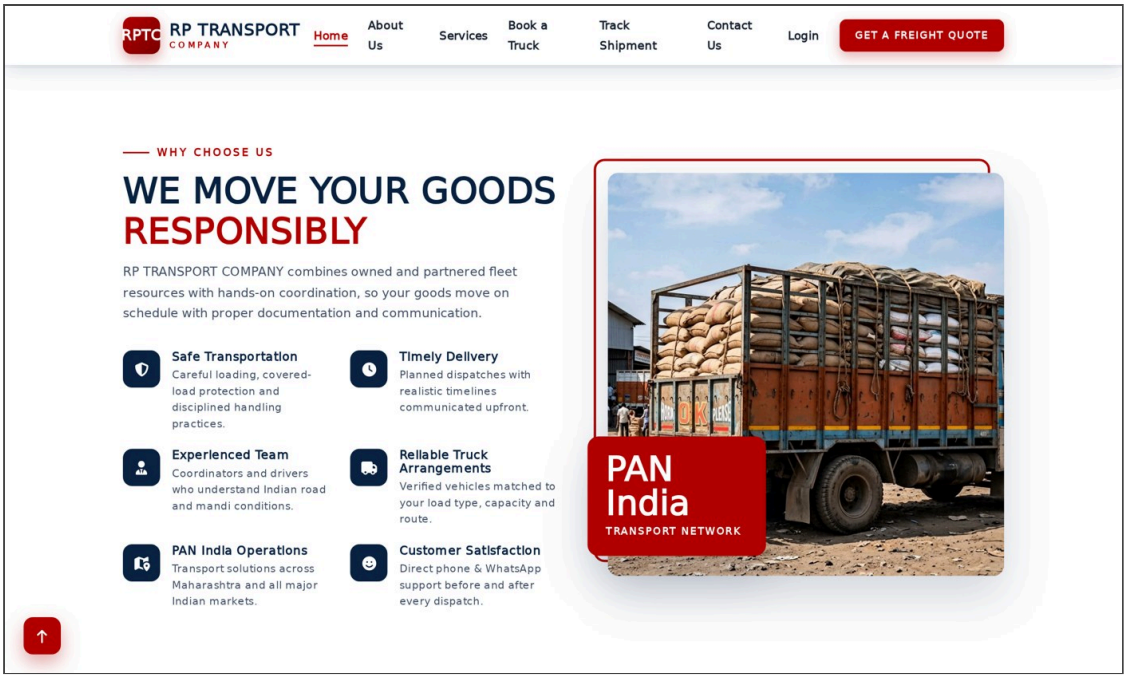


Figure 5.9 — "We move your goods responsibly" section with the eight reasons to choose RPTC

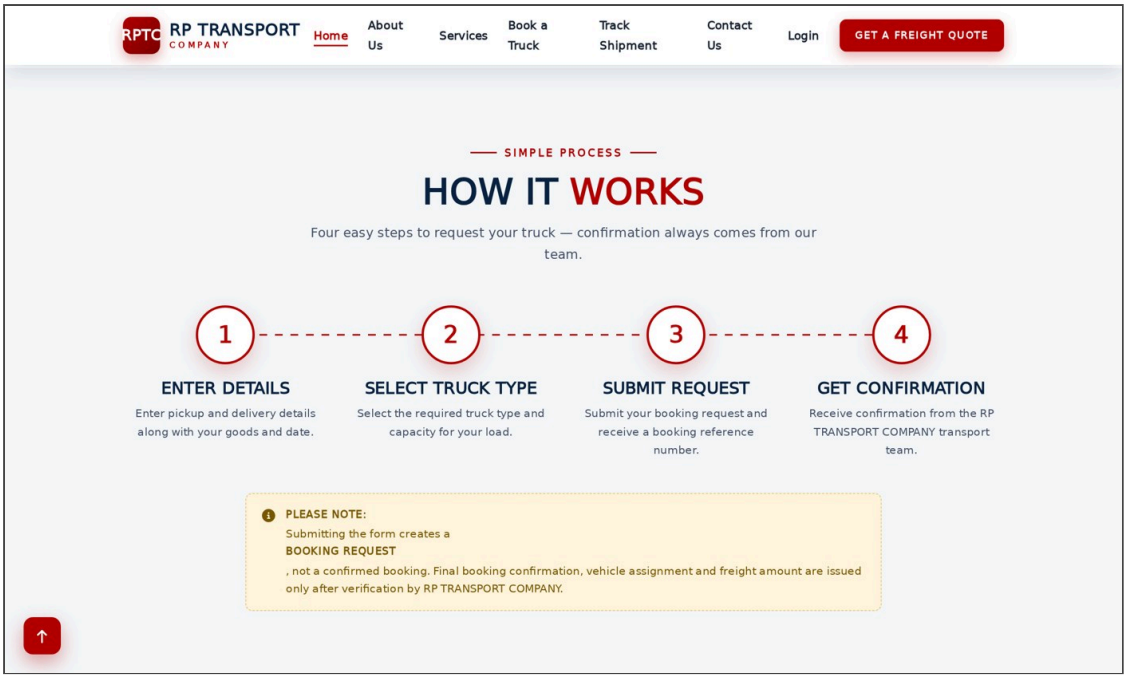


Figure 5.10 — The four-step booking process explained to the first-time visitor

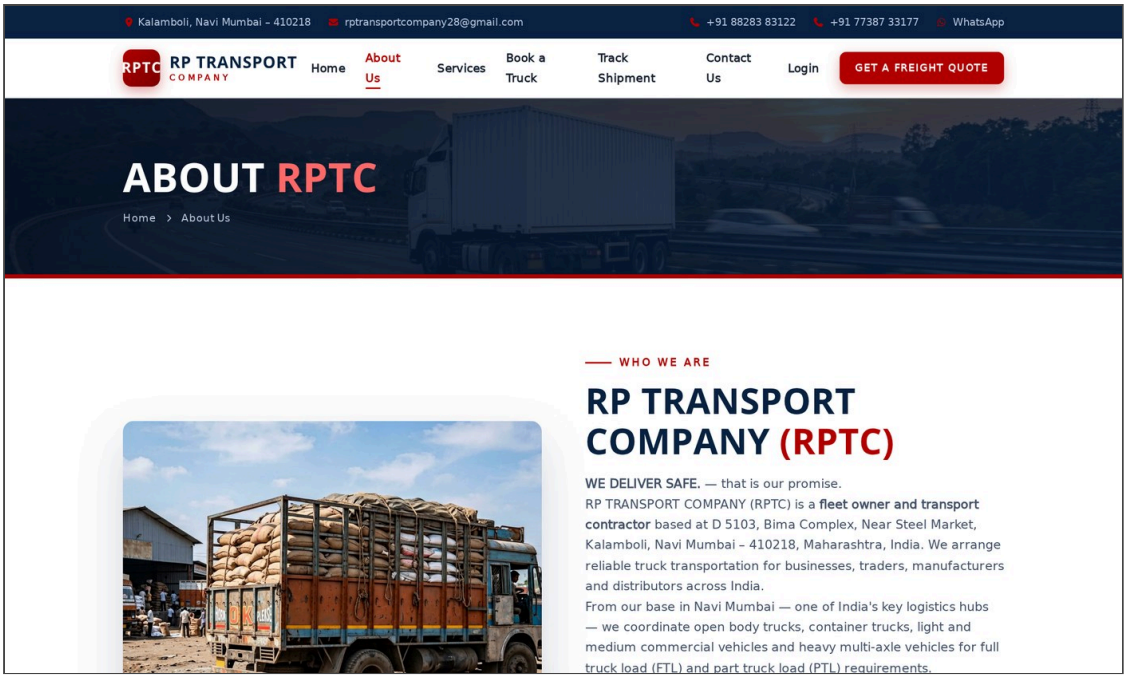


Figure 5.11 — About Us page: company introduction, mission, vision and values

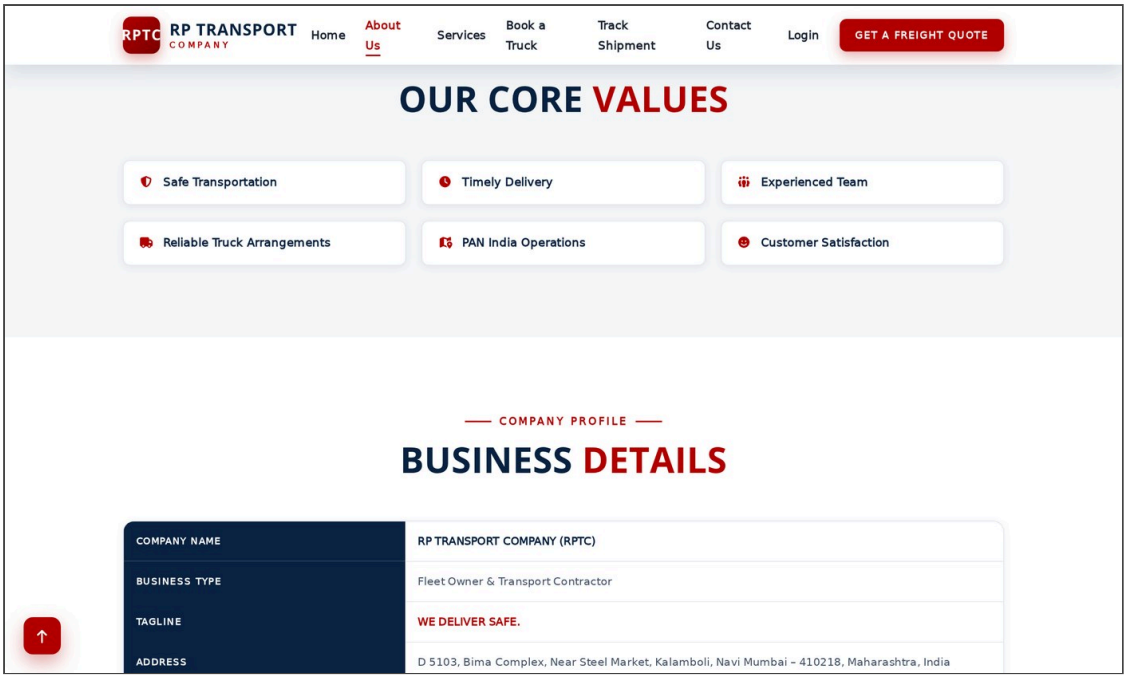


Figure 5.12 — Core values grid and the business details table of the company profile

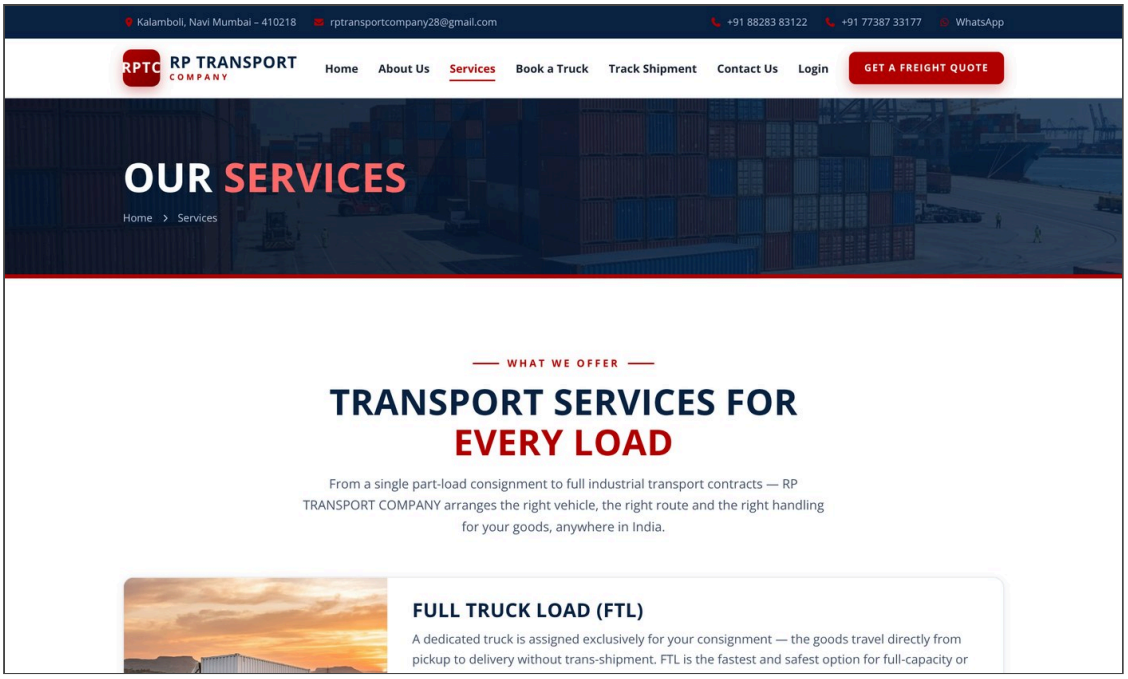


Figure 5.13 — Services page: detailed description of each service with its feature list and booking link

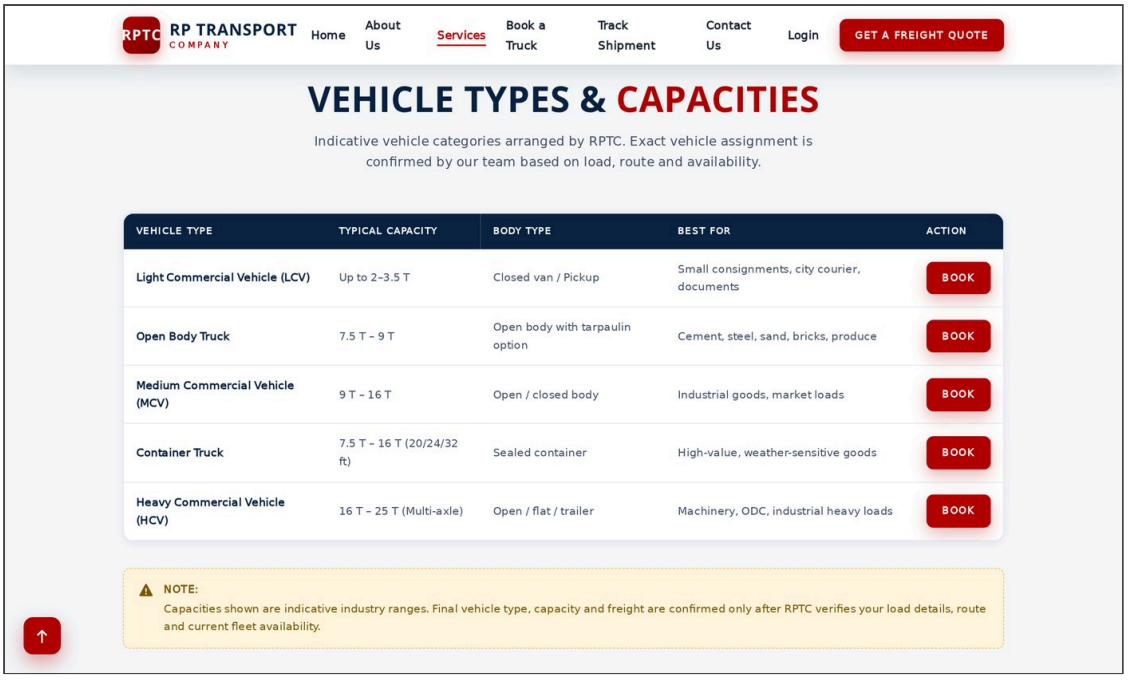


Figure 5.14 — Vehicle types and capacities table, with an indicative capacity note

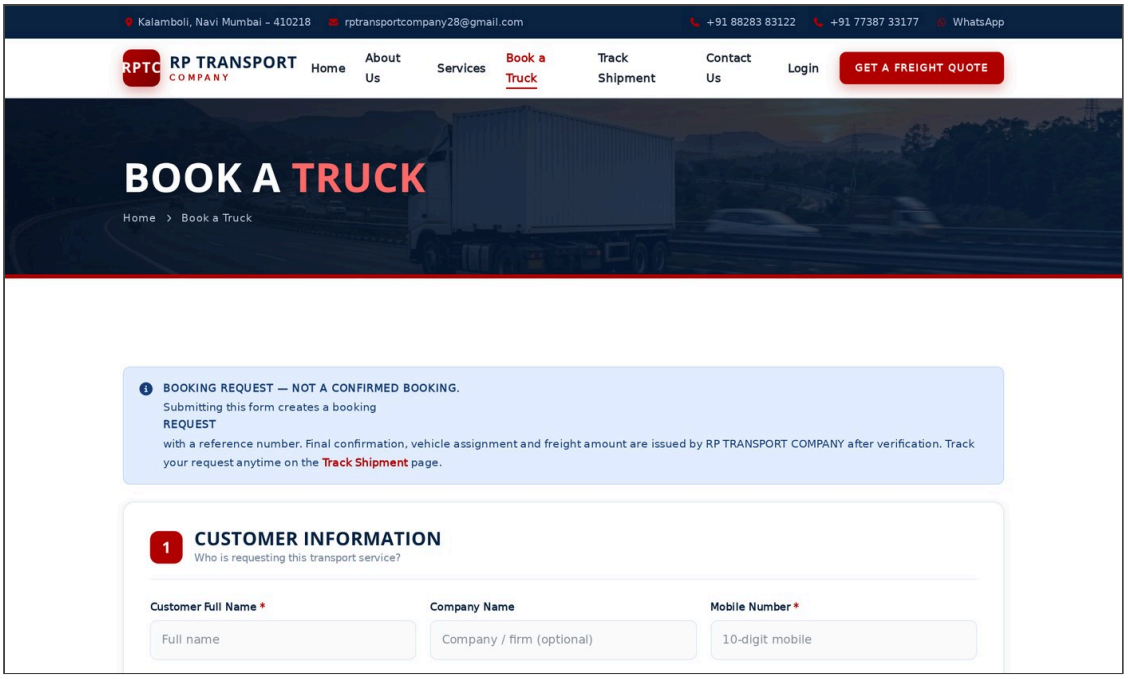


Figure 5.15 — Online truck booking form, section 1 of 6; the banner states that the form creates a request, not a confirmed booking

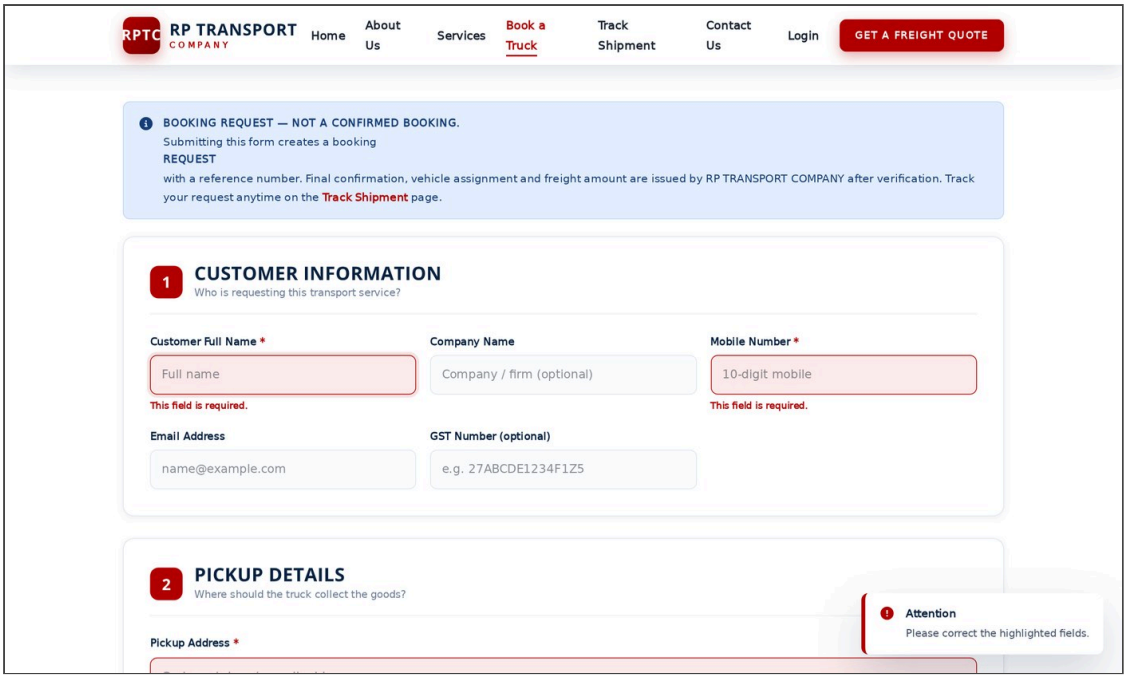


Figure 5.16 — Client-side validation: the empty mandatory fields are outlined and explained, and a toast summarises the problem

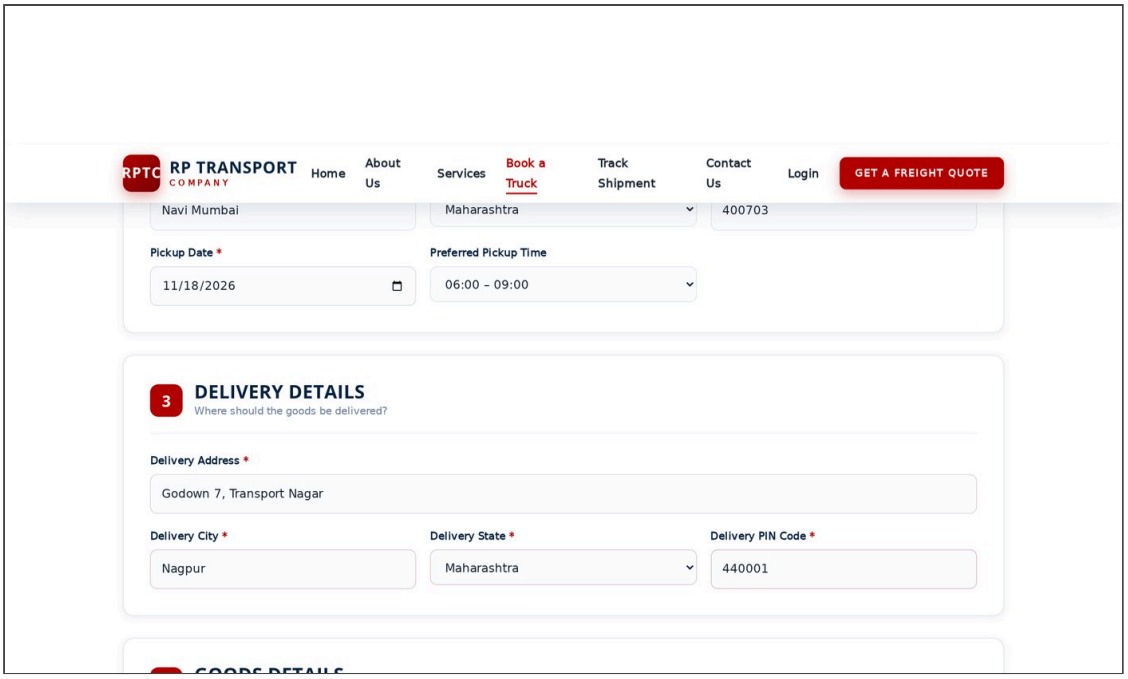


Figure 5.17 — A completed booking form ready for submission, carrying a real-looking route, load and vehicle selection

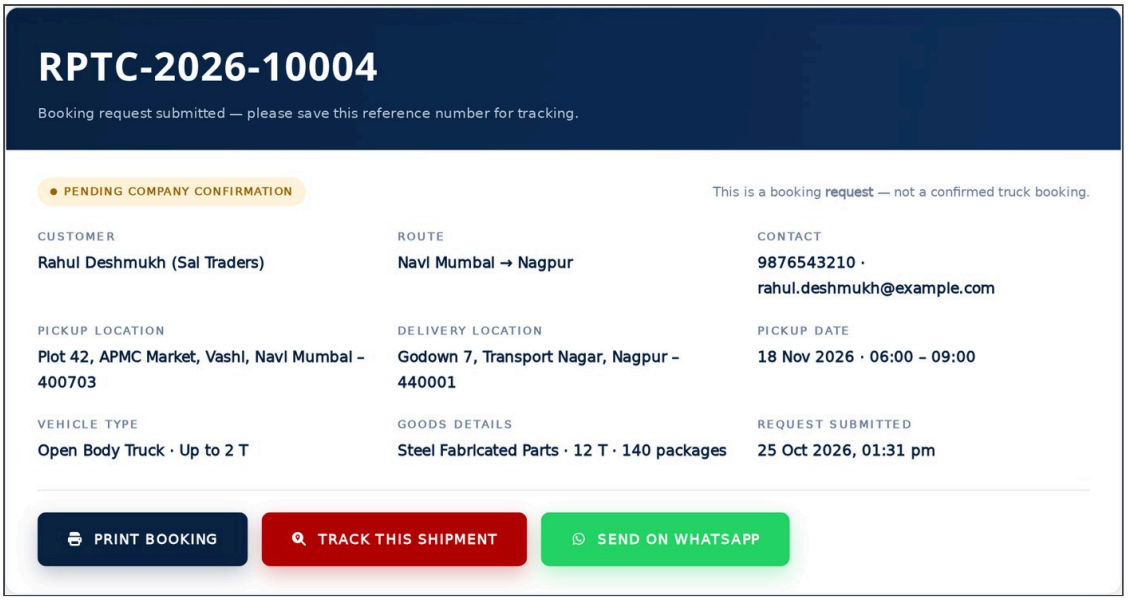


Figure 5.18 — Generated booking summary with the reference number RPTC-2026-10004 and the Print, Track and WhatsApp actions

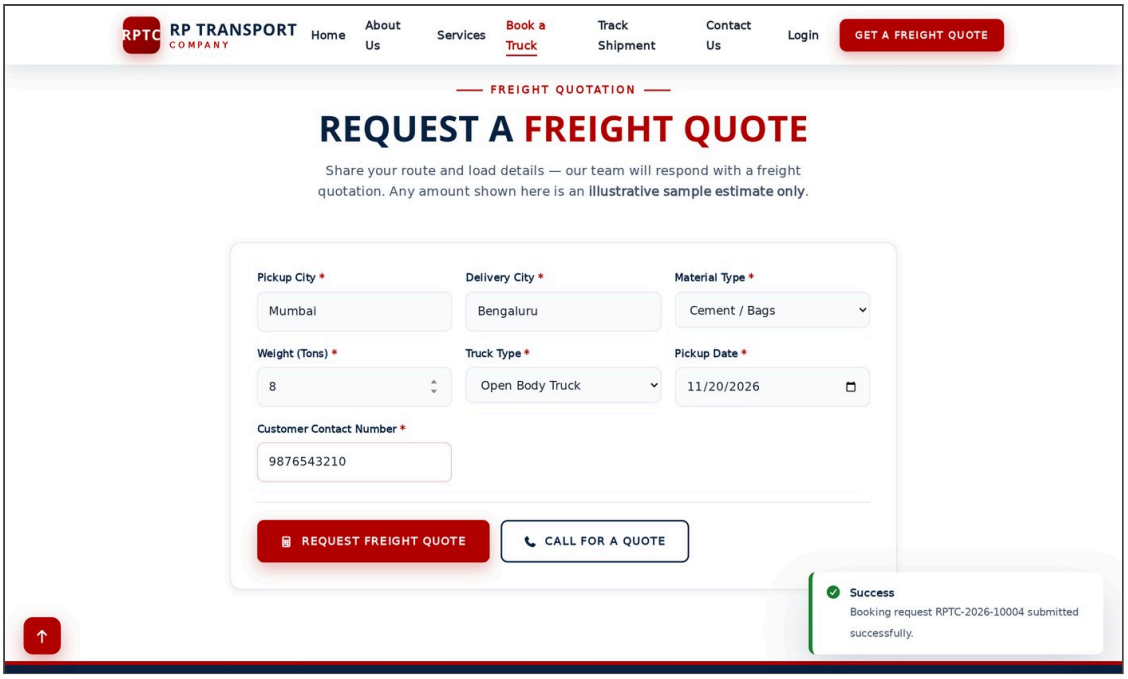


Figure 5.19 — Freight quotation request form on the Book a Truck page

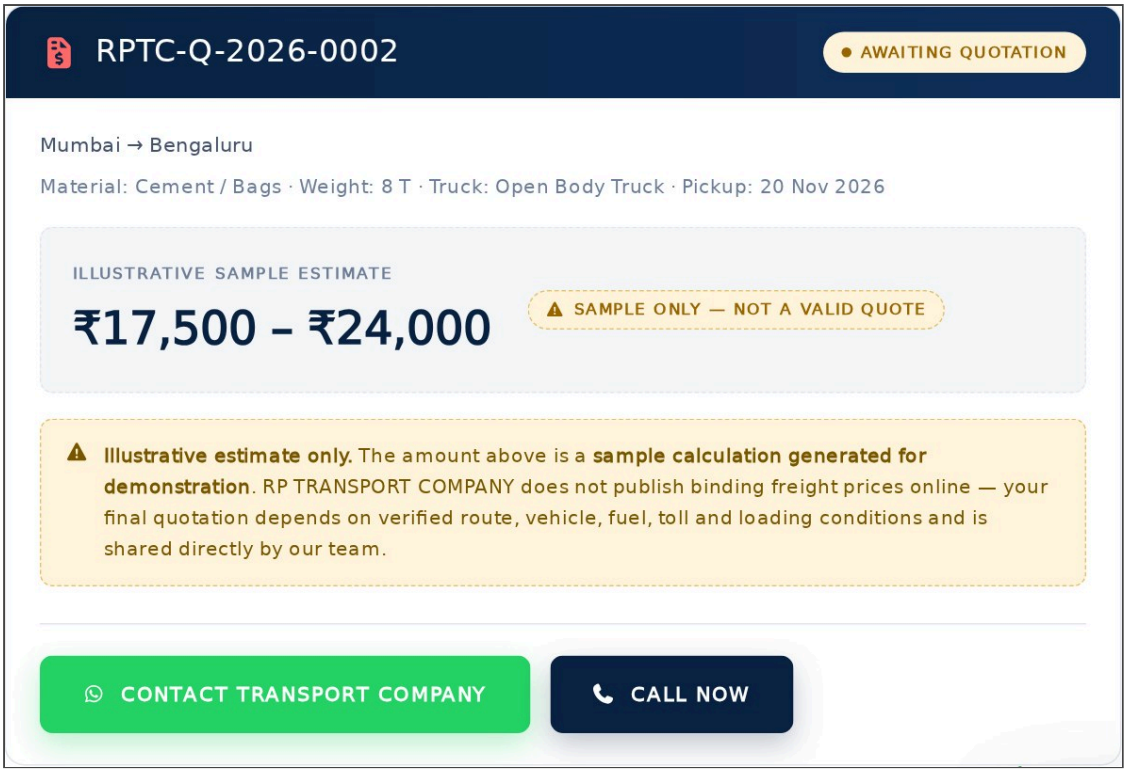


Figure 5.20 — Illustrative freight estimate produced for a quotation request, marked “sample only — not a valid quote”

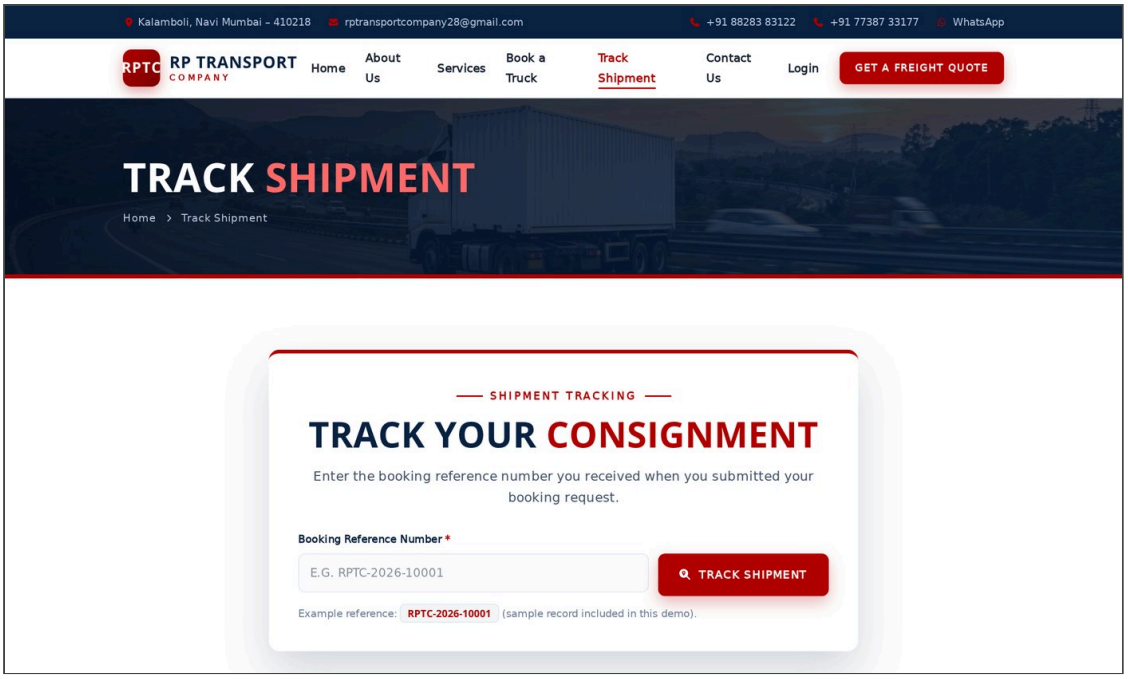


Figure 5.21 — Shipment tracking page: lookup by booking reference, with the sample reference shown as an example

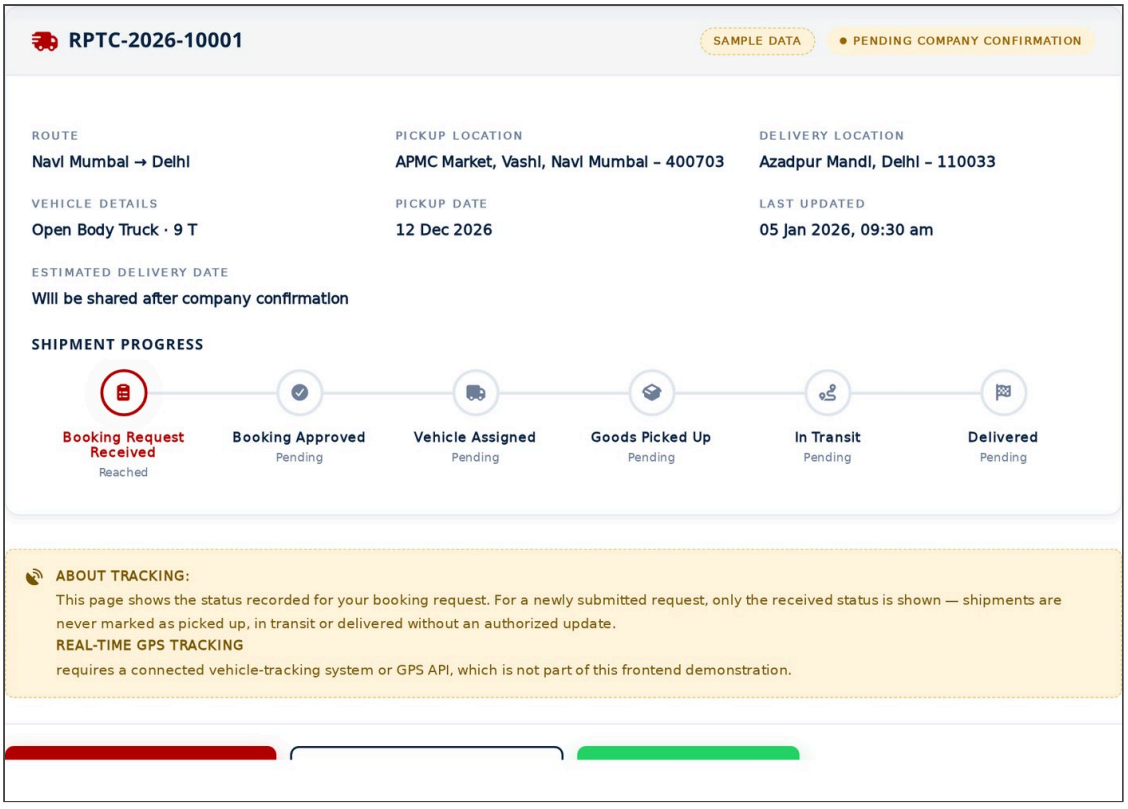


Figure 5.22 — Tracking result for RPTC-2026-10001: route, vehicle, dates and the six-stage progress timeline

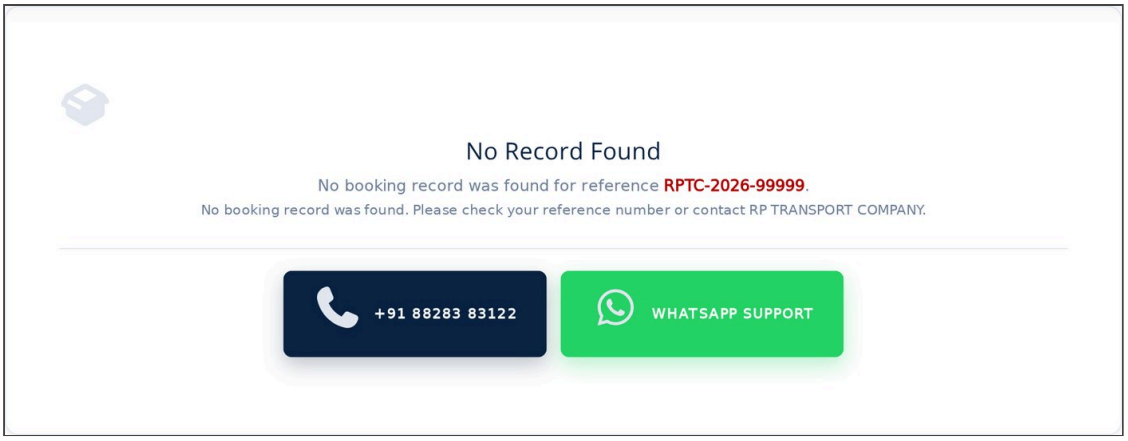


Figure 5.23 — The "No Record Found" state, with the support alternatives offered instead of a dead end

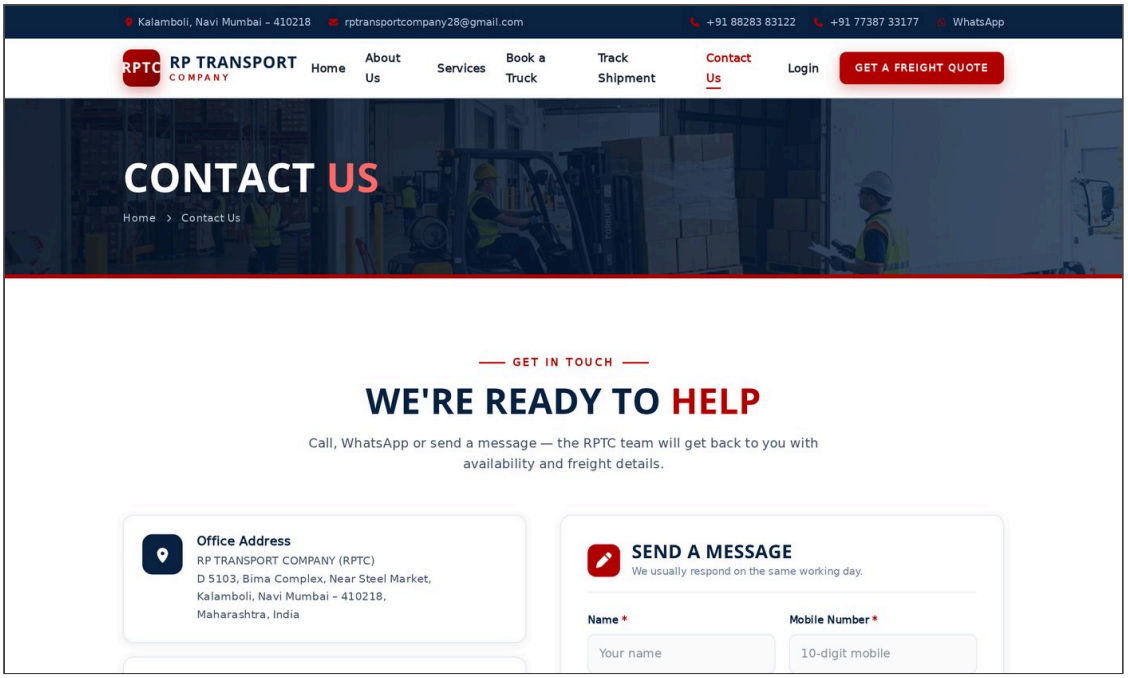


Figure 5.24 — Contact Us page: address, click-to-call numbers, WhatsApp, e-mail and operating area

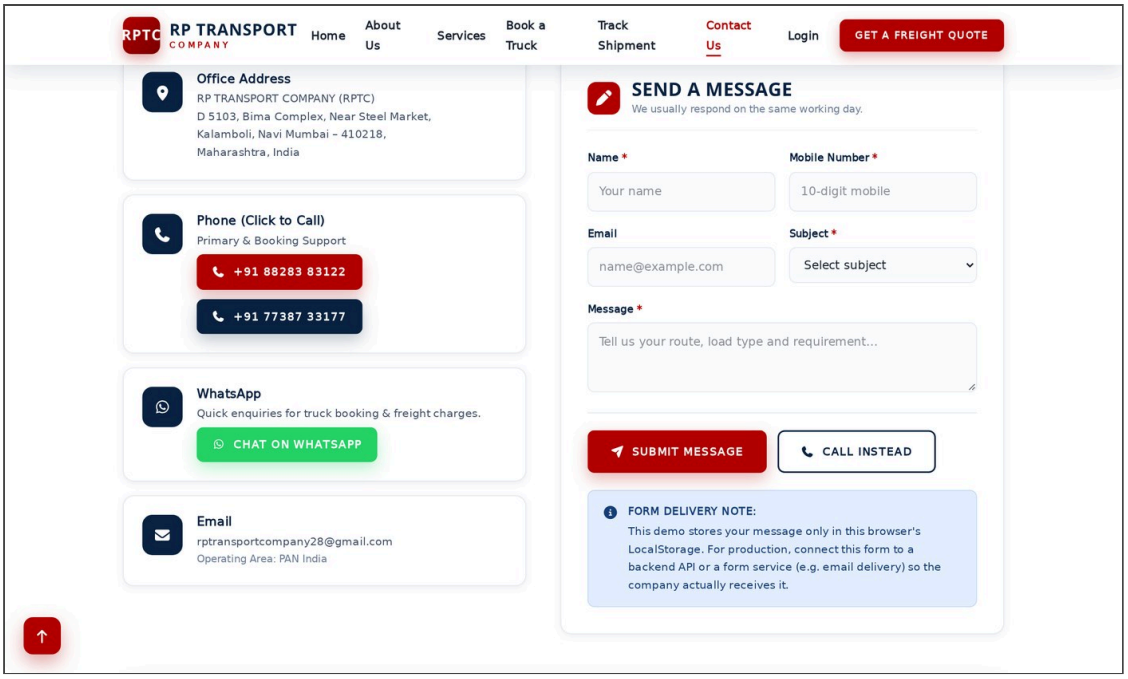


Figure 5.25 — Validated contact form with the enquiry subject list and the Map of the Kalamboli area

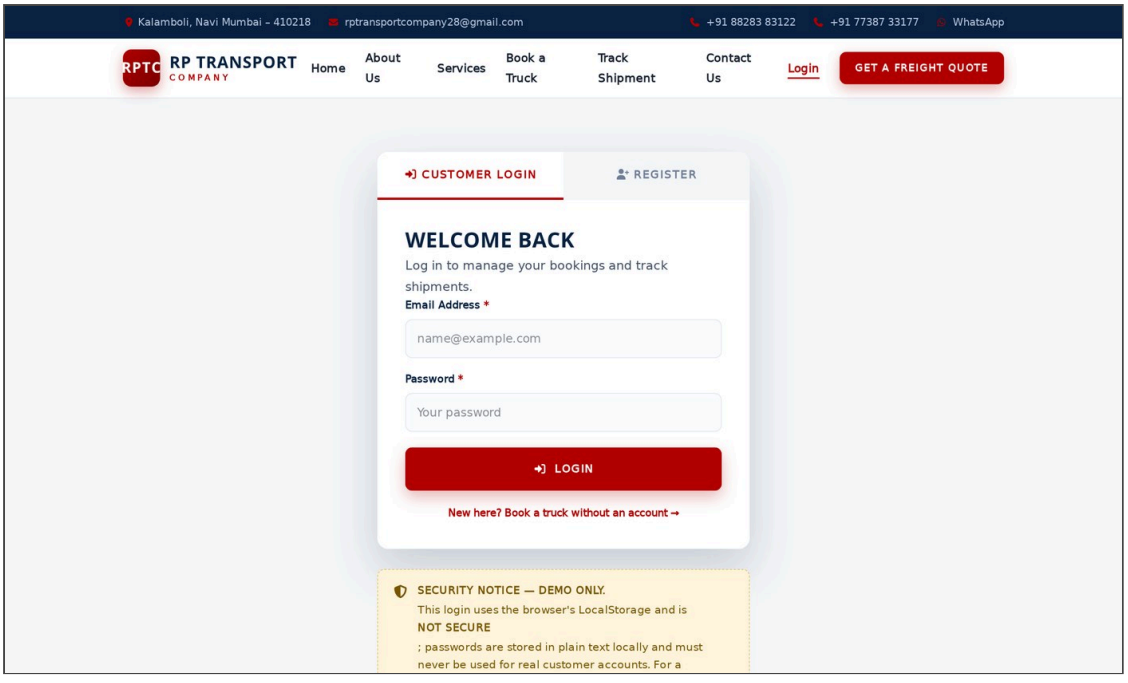


Figure 5.26 — Customer login and registration page, with the security notice displayed openly

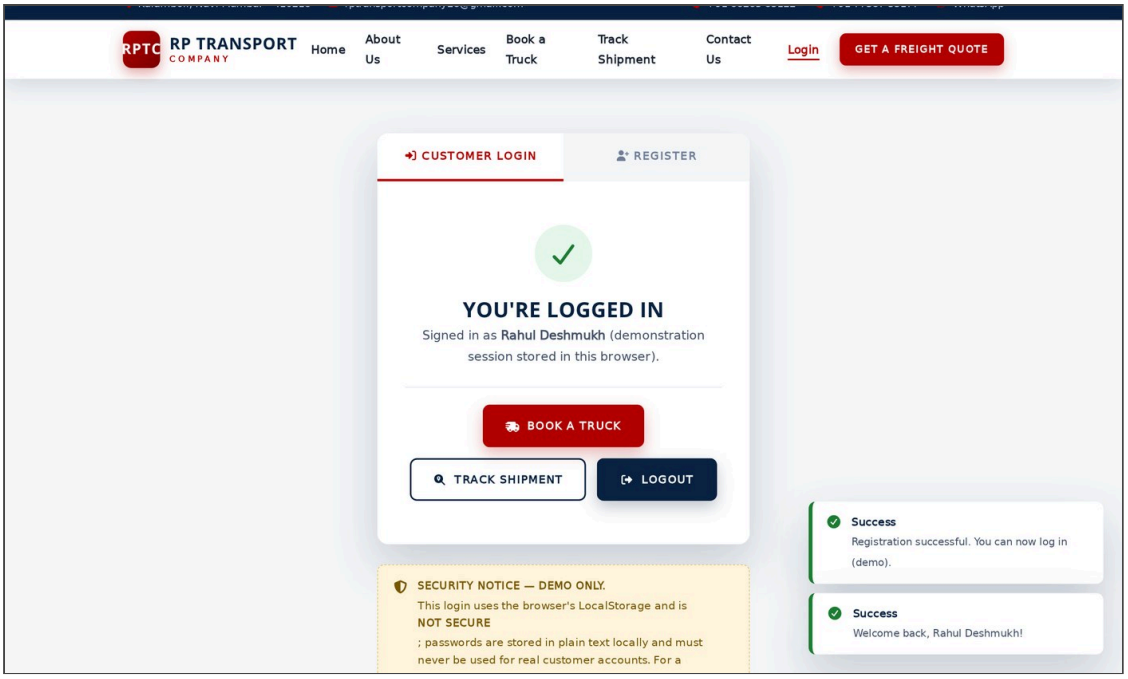


Figure 5.27 — A registered customer signed in; the session is a local demonstration only

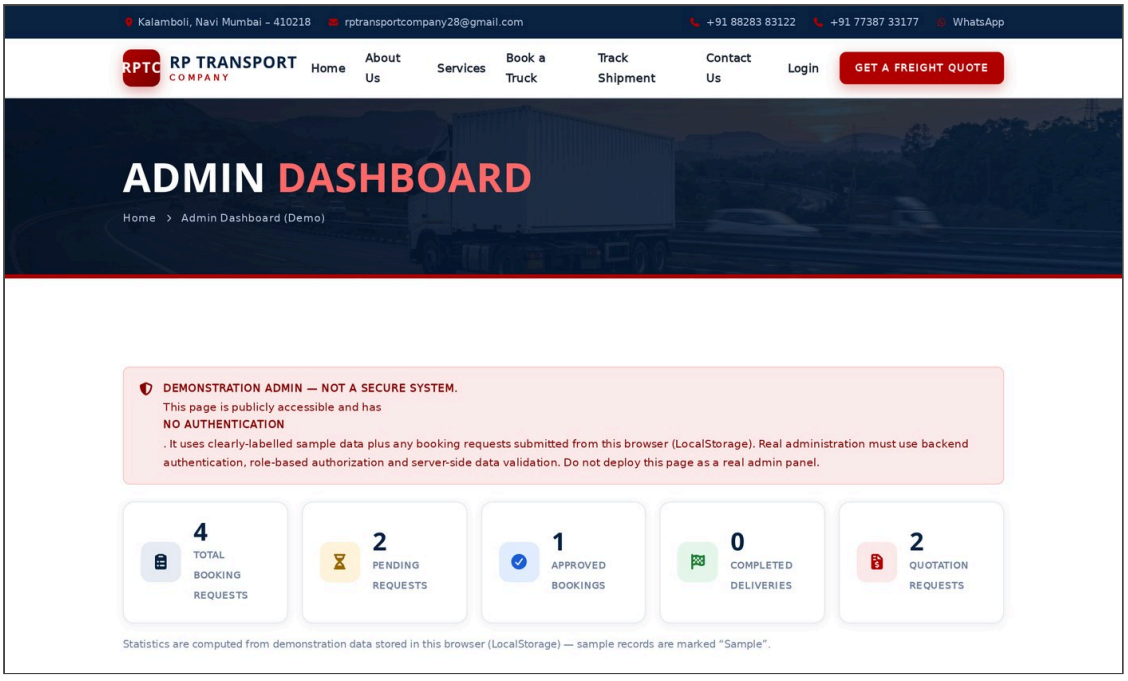


Figure 5.28 — Admin dashboard: demonstration notice and the five statistics cards computed from the stored records

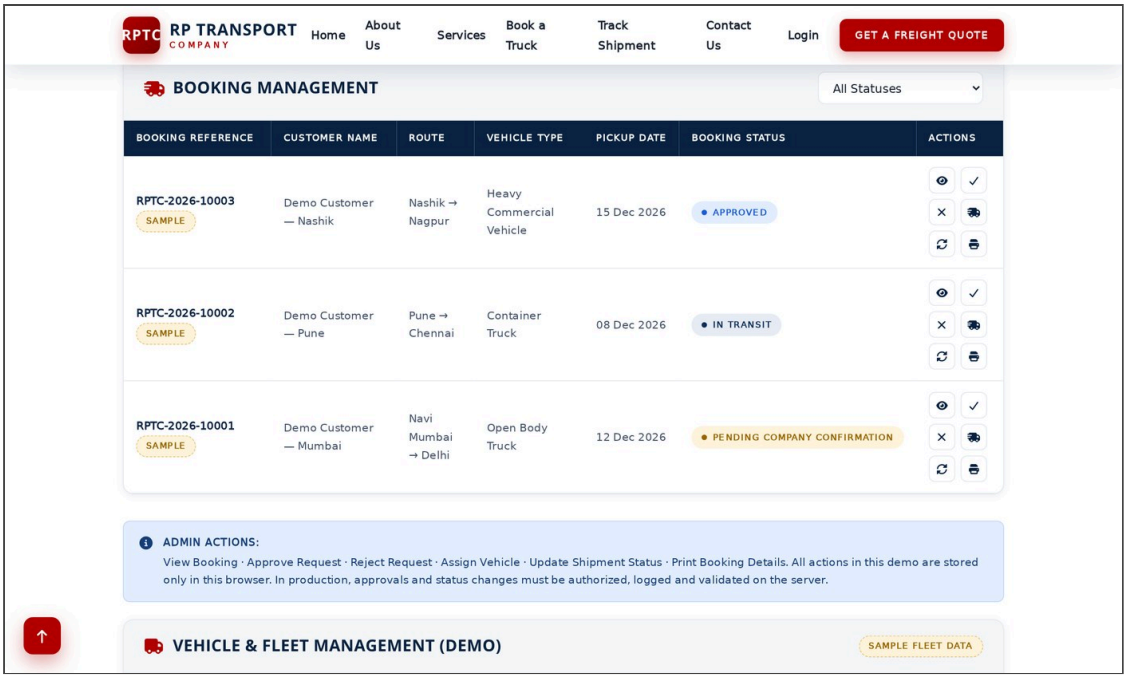


Figure 5.29 — Booking management table with status pills, sample tags and the six actions available for each request

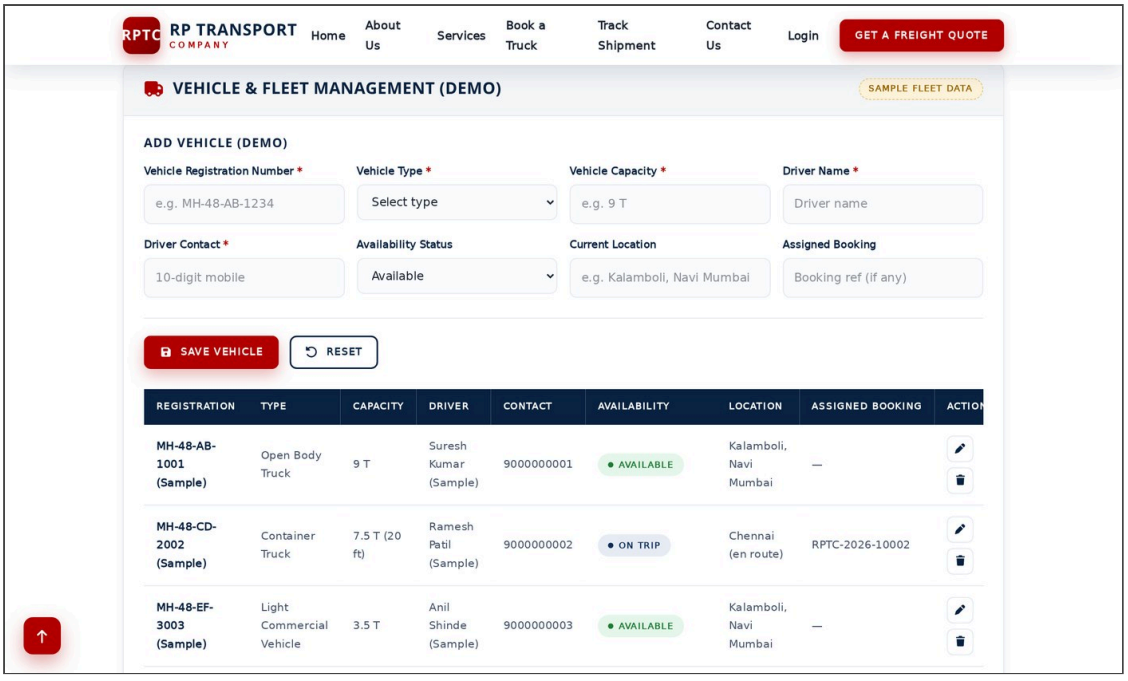


Figure 5.30 — Vehicle and fleet management: add-vehicle form and the fleet register with availability status

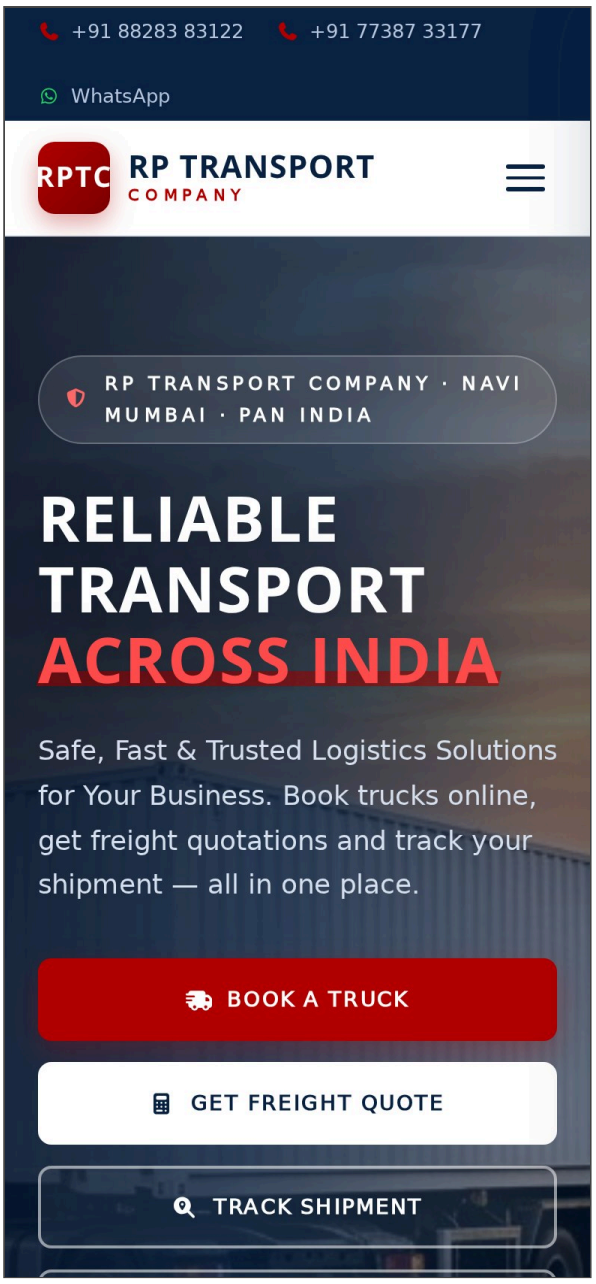


Figure 5.31 — Responsive mobile layout at 390 × 844 pixels, with the hamburger navigation

6. SYSTEM TESTING

System testing is the stage before system implementation where the system is made error free and all the needed modifications are made. The system was tested with test data and necessary corrections to the system were carried out. All the screens were checked against the requirements recorded in Chapter 3 and approved. The system was kept user friendly, with inline help messages and toast notifications to assist the user wherever necessary — for example the *booking request is not a confirmed booking* banner, the field-level error messages and the explicit "simulated result" and "sample only" labels.

6.1 Test Plan

A test plan is a general document for the entire project which defines the scope, the approach to be taken and the schedule of testing, as well as identifying the test items for the entire testing process and the personnel responsible for the different activities of testing. This document describes the plan for testing the transport management website.

Major testing activities were:

- Test units
- Features to be tested
- Approach for testing
- Test deliverables
- Schedule
- Personnel allocation

Features to be tested. Navigation and responsive layout; the quick availability form; the six-section booking form and its validation; reference-number generation; the booking summary and print output; the freight quotation form and its estimate; tracking of a valid, an invalid and a newly-created reference; the contact form; registration and login; all nine administrator actions (view, approve, reject, assign vehicle, update status, print, add, edit and delete vehicle); the WhatsApp, telephone and e-mail links; and behaviour with JavaScript errors cleared from the console.

Approach. Testing was carried out as black-box functional testing on the running website in Google Chrome, Microsoft Edge and Mozilla Firefox, with the browser developer tools open so that every console error and failed network request was noticed. Each feature was first tested with valid data (positive testing), then with invalid, empty and boundary data (negative testing), and finally with data entered in an unexpected order — for example pressing *Submit* without touching any field, or refreshing the page after a booking had been saved. Responsive behaviour was tested with the

device-emulation mode at 1440, 1024, 768 and 390 pixels, and once on a physical Android phone. Data persistence was tested by closing and reopening the browser.

Schedule and personnel. Testing was carried out in the last three weeks of the project: unit testing of each page in the first week, integration testing of the booking-to-admin-to-tracking flow in the second week and regression testing together with the demonstration to the guide in the third week. The student performed the testing and recorded the results; the project guide reviewed the test cases and the demonstration.

6.2 Test Units and Levels of Testing

Test case specification is a major activity in the testing process. In this project two levels of testing were performed:

- Unit testing
- System testing

The basic units in unit testing are:

- Validating the user request
- Validating the input given by the user
- Exception handling

Each unit was tested in isolation. The validation unit was tested with an empty form, a form with a nine-digit mobile number, a form with an invalid e-mail address, a form with a five-digit PIN code and a form with a weight of zero; each case produced the intended field-level message and no JavaScript error. The reference generator was tested by submitting several bookings in one session and confirming that the sequence advanced without repetition (RPTC-2026-10001, 10002, 10003, 10004). Exception handling was tested by searching for a reference that does not exist, by opening a page whose data store had never been seeded, and by clearing LocalStorage while the site was open; in every case the page degraded to an empty state or a clear message instead of failing.

The basic units in system testing are:

- Integration of all programs is correct or not
- Checking whether the entire system after integrating is working as expected
- The system is tested as a whole after the unit testing

Integration was verified along the complete business path: a booking submitted on *booking.html* appeared in the admin booking table with the status *Pending Company Confirmation*; approving it changed the status pill; assigning a vehicle from the fleet wrote the registration number onto the booking and marked the vehicle *On Trip*; advancing the status to *In Transit* moved the tracking timeline two stages forward; and printing the booking produced a clean document with no navigation

or buttons. The quotation path was verified in the same way, from request to illustrative estimate to appearance in the quotation counter of the dashboard.

6.3 Other Testing Strategies

Alpha Testing. This was done at the developer's site by a customer. The software is used in a natural setting with the developer “looking over the shoulder” of the user and recording errors and usage problems; alpha tests are conducted in a controlled environment. In this project the alpha test was the demonstration given to the project guide, during which the guide attempted a booking and a tracking lookup unaided. Two usability problems were found and corrected as a result: the tracking page originally gave no suggestion when a reference was not found, and the booking form did not warn that submission creates a request rather than a booking. Both corrections are visible in the final screens (Figures 5.15 and 5.23).

Beta Testing. This was conducted at one or more customer sites by the end-user of the software. Unlike alpha testing, the developer is generally not present; therefore the beta test is a “live” application of the software in an environment that cannot be controlled by the developer. The customer records all problems encountered during beta testing and reports them to the developer at regular intervals. As a result of problems reported during beta tests, software engineers make modifications and then prepare for release of the software product to the entire customer base. For this project the beta test was a short trial by two traders of the Kalamboli area and by one driver, on their own Android phones. Their observations — that the booking form was long, that the mobile menu was hard to notice and that the driver needed a printable slip — led to the quick booking form on the home page, the simplified hamburger button and the Print Booking action.

Compatibility and responsive testing. The site was checked on Chrome, Edge and Firefox on Windows, on Chrome on Android and on Safari on iOS. Because it uses only standard HTML, CSS and JavaScript and no build step, no browser-specific code was needed; the only browser-dependent behaviour is the appearance of the native date picker, which was verified to remain usable in all three engines.

Accessibility and console checks. Every form was operated with the keyboard alone, every icon-only button carries an *aria-label*, and the console was required to be free of errors on all eight pages in the final build.

6.4 Test Cases Executed

Table 6.1 — Test Cases and Results

ID	Test case	Input / action	Expected result	Result
T-01	Page navigation	Click each of the eight menu links	The correct page opens; the active link is highlighted	Pass

ID	Test case	Input / action	Expected result	Result
T-02	Mobile menu	Open the site at 390 px and tap the hamburger button	The menu slides in with an overlay; tapping a link closes it	Pass
T-03	Empty quick-booking form	Press “Check Truck Availability” without filling any field	All mandatory fields outlined; toast asks to correct the fields	Pass
T-04	Invalid mobile number	Mobile = 98765 (five digits)	Message “Enter a valid 10-digit Indian mobile number”	Pass
T-05	Invalid PIN code	PIN = 4007	Message “Enter a valid 6-digit PIN code”	Pass
T-06	Invalid e-mail	E-mail = rahul.example.com	Message “Enter a valid email address”	Pass
T-07	Zero weight	Weight = 0	Message “Enter a value greater than zero”	Pass
T-08	Past pickup date	Date picker opened	Dates before today are not selectable	Pass
T-09	Availability preview	Valid quick-booking form, submitted	Indicative vehicle list shown with a “Simulated result” pill and warning	Pass
T-10	Booking submission	Complete six-section form, submitted	Reference RPTC-2026-10004 generated; summary displayed	Pass
T-11	Reference sequence	Submit four bookings in one session	References increase 10001, 10002, 10003, 10004 without repetition	Pass
T-12	Print booking	Press “Print Booking” on the summary	Print preview shows the summary only — no menu, banner, buttons or toasts	Pass
T-13	Quotation request	Route Mumbai to Bengaluru, 8 T, open body truck	Reference RPTC-Q-2026-0002 with an illustrative range marked “sample only”	Pass
T-14	Tracking — valid reference	RPTC-2026-10001	Full record with the six-stage timeline, stage 1 marked reached	Pass
T-15	Tracking — unknown reference	RPTC-2026-99999	“No record found” panel with call and WhatsApp alternatives	Pass
T-16	Tracking — empty reference	Field left blank, submitted	Toast “Please enter a booking reference number”; no result panel	Pass
T-17	Tracking after status change	Admin moves the booking to In Transit, then track it	Timeline shows stages 1–5 progressed, stage 6 pending	Pass

ID	Test case	Input / action	Expected result	Result
T-18	Contact form	Valid enquiry, then an empty enquiry	Success message and local record; empty form rejected with messages	Pass
T-19	Registration	Register with a duplicate e-mail, then with mismatched passwords	Duplicate blocked with a message; password mismatch reported	Pass
T-20	Login and logout	Login with the registered credentials; then press Logout	Signed-in panel appears; logout clears the session and restores the forms	Pass
T-21	Admin — dashboard counts	Open the dashboard after two bookings and one quotation	Counters show 4 total, 2 pending, 1 approved, 0 completed, 2 quotations	Pass
T-22	Admin — approve	Press the approve action on a pending booking	Status becomes APPROVED; history entry added; counters update	Pass
T-23	Admin — reject	Press reject and confirm in the modal	Status becomes REJECTED; tracking page shows the rejection notice	Pass
T-24	Admin — assign vehicle	Select vehicle V-001 for a booking	Registration written on the booking; vehicle marked On Trip with the reference	Pass
T-25	Admin — add vehicle	Add MH-48-XY-9009, open body, 9 T	Row appears in the fleet table and in the assignment list	Pass
T-26	Admin — edit / delete vehicle	Edit capacity, then delete the new vehicle	Record updated, then removed after confirmation	Pass
T-27	Filter	Choose “Approved” in the status filter	Only approved bookings are listed	Pass
T-28	Data persistence	Submit a booking, close the browser, reopen and refresh	Booking still present in the dashboard and trackable	Pass
T-29	External links	Press the phone, WhatsApp and e-mail actions	Dialler, WhatsApp chat and mail composer open with the correct number and message	Pass
T-30	Console cleanliness	Visit all eight pages with the console open	No JavaScript errors and no failed local requests	Pass

All thirty test cases were executed and passed after the corrections described in Section 6.3. The two defects found during alpha testing were usability defects, not functional ones, and both were closed before the final build.

6.5 Test Deliverables

The following documents are required besides the test plan:

- Unit test report for each unit
- Test case specification for system testing
- The report for system testing
- Error report

The test case specification for system testing has to be submitted for review before the system testing commences. In this project the unit test reports were maintained page-wise, the test case specification is reproduced as Table 6.1, and the error report consists of the two alpha-test observations together with their corrections. All of these were reviewed by the project guide before the final demonstration.

7. IMPLEMENTATION AND EVALUATION

During the software-testing phase each module of software is thoroughly tested for bugs and for accuracy of output. The system developed is very user friendly and the detailed documentation is also given to the user as online help wherever necessary — in this project that online help takes the form of the demonstration notice on the booking page, the security notice on the login page, the simulation warning under the availability result, the “sample only” label on the freight estimate, and the explanatory panel under the tracking timeline. The implementation phase normally ends with the formal test involving all the components, which is the test run recorded in Chapter 6.

7.1 Implementation of the Modules

The entire system was developed using HTML5, CSS3 and JavaScript, with the browser's LocalStorage as the demonstration data store. HTML is used to design the pages and to declare the structure of every form; CSS3 is used for the corporate visual language, the two typefaces, the grid layouts and the responsive breakpoints; JavaScript is used for client-side validation so that the user can enter only appropriate input in the input fields, for generating reference numbers, for writing and reading records, and for rendering the booking summary, the tracking timeline and the admin tables.

The implementation followed the module decomposition fixed in Chapter 2. The public module consists of the six informational and transactional pages. The customer module consists of the registration and login forms and the session panel. The administrator module consists of the dashboard with its statistics, booking table, action modals and fleet register. Because each page initialises only the parts of the script that its own markup requires, the same script file is safely shared by all eight pages, and no page carries code that it does not use.

The most carefully implemented part is the booking pipeline, because it is the part a customer actually depends on. A booking passes through four steps inside the code:

1. **Collection.** The six form sections are read into a single object with nested groups for customer, pickup, delivery, goods, vehicle and payment. Fields arriving from the home page quick form are pre-filled from the query string, so the customer does not type the route twice.
2. **Validation.** Every element carrying the *required* attribute is checked, together with the format rules for the mobile number, the e-mail address, the PIN code and the numeric minimum. The first invalid field receives focus, and each invalid group shows its own message.
3. **Identification.** A reference number is generated by scanning the existing records for the highest sequence of the current year and adding one, which guarantees uniqueness without a server-side counter.
4. **Presentation.** The record is stored, the summary is rendered from the same object that was stored (so the summary can never disagree with the data), the form is reset, and the customer is

offered print, tracking and WhatsApp actions.

The administrator module reuses the same record structure. Approving, rejecting, assigning a vehicle and advancing the status all pass through a single *updateBooking()* function which writes the new status, appends a timestamped entry to the status history, and then re-renders the statistics and the booking table. This single point of update is what keeps the dashboard, the tracking page and the printed document consistent with each other.

7.2 What is Real and What is Demonstration

An honest evaluation of the delivered system must separate the parts that are fully working from the parts that are illustrative. This separation is also built into the interface, so that no visitor is misled.

Fully working in the delivered system	Demonstration only
All page navigation, mobile hamburger menu and responsive layout	The “Check Truck Availability” result on the home page is simulated; it does not confirm a real truck
Form validation with inline messages and toast notifications	The freight estimate is an illustrative sample calculation, not a valid quotation
Booking reference generation in the form RPTC-YYYY-NNNNN	Booking approvals, rejections and status changes are demonstration actions stored in the browser
Booking submission, summary rendering and printing of the booking document	Shipments are never auto-marked as picked up, in transit or delivered; only an authorised update changes the status
Quotation request capture and the illustrative estimate display	The testimonials on the home page are marked sample content; no verified customer counts are claimed
Tracking lookup with the six-stage timeline and the not-found state	Statistics on the dashboard are computed from demonstration records only
Contact form validation, click-to-call, WhatsApp and e-mail links, embedded map	The contact form stores the message locally; it does not deliver e-mail anywhere
Registration, login, session display and logout	Authentication is local and unencrypted; the admin page has no login at all
Admin booking and fleet management with status history	All records live in one browser and one device; nothing is shared between users

7.3 Evaluation

Evaluated against the four questions recorded in Section 3.1, the system meets its objective. A customer can now describe a load once, in a structured form, and receive a reference number immediately; the enquiry can be found again by that number; the office can see what is pending and which vehicle is free; and the customer can follow the consignment without telephoning the office.

The interface is simple enough that the two traders who tested it completed a booking unaided on the first attempt.

The evaluation also has to record what the system cannot yet do. It is a front end. It does not authenticate anybody securely, it does not share data between users or devices, it does not send e-mail or SMS, it does not calculate a real freight, and it does not know where a truck physically is. These are not defects of implementation; they are the boundary of a front-end project, and the design has been arranged so that each of them can be added behind the existing screens without redesigning the interface. During the software-testing phase each module was checked for both functional correctness and the accuracy of its output, and at the end of the implementation the whole system was tested as one integrated application rather than as eight separate pages.

7.4 Production Upgrade Path

The following components are recommended before the website is used for real business. They are listed in the order in which they would normally be built.

- A backend API for bookings, quotations, customers, vehicles, drivers and shipments, replacing the *getData()* and *setData()* helpers with HTTP calls.
- A relational or document database (PostgreSQL, MySQL or MongoDB) following the schema of Tables 5.2 to 5.6, with scheduled backups and error logging.
- Secure authentication for customers and administrators — hashed passwords, server-side sessions or JWT, HTTPS, and role-based access control on the dashboard.
- Server-side validation of every submitted booking, so that the client-side rules cannot be bypassed.
- A verified fleet availability table and an approval workflow, so that the availability result becomes a real answer.
- A pricing rule set maintained by the office, so that a freight quotation can be issued automatically and printed.
- Shipment status history with authorised updates and an audit log, and e-mail and SMS notifications on confirmation and on each stage change.
- A payment gateway, if online advance payment is to be collected.
- GPS integration through a vehicle-tracking provider for real-time position.
- Secure document management for invoices, loading slips and e-way bills.

8. CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

This project has given me an ample opportunity to design, code, test and implement an application. It has helped in putting into practice various software engineering principles and database management concepts, such as maintaining the integrity and consistency of data, separating the logical design of a record from its physical storage, and verifying a design against recorded requirements. It has also helped me to learn more about HTML5, CSS Grid and Flexbox, modern JavaScript, the Web Storage API, responsive design, accessibility and the discipline of writing documentation that matches the delivered code.

The outcome is a complete, working front end for RP TRANSPORT COMPANY: eight linked pages that present the company and its ten services, accept a full truck booking request with validation and a generated reference number, capture freight quotation requests, let a customer track a consignment through six stages, register and log in, contact the office, and let the office manage bookings and the fleet from a single dashboard with a status history. Thirty test cases were executed and passed, and every feature that is illustrative rather than real is labelled as such inside the interface.

I thank my guide for her invaluable contribution in guiding me throughout the project. I also thank my parents and friends who have supported and motivated me to complete this project successfully.

8.2 Extensibility

The other features that a transport management service normally provides can also be incorporated into this system. The encryption standards can be used to make the transactions more secure, and the Secure Socket Layer protocol can be used in implementing the system, which gives the highest security over the Internet. Because the presentation layer is separated from the data layer — the pages read and write through two small helper functions and a single key object at the top of the script — a backend can be introduced without touching the screens. Additional modules such as driver management, invoice and e-way-bill generation, a customer dashboard listing past bookings, an SMS gateway, a rating and feedback module, and a reporting module for monthly freight and tonnage can be added as new pages that reuse the same layout, components and validation rules.

8.3 Future Enhancement

In the present version the availability of trucks is simulated and the freight figure is illustrative, because a real answer needs a verified fleet database and the firm's own pricing rules. My first future enhancement is therefore to connect the website to a backend with a real fleet and driver register, so that availability and freight become accurate and every booking is confirmed by the office from any device rather than from one browser.

The second enhancement is real-time tracking. At present the timeline reflects the status recorded by the office. By integrating a GPS vehicle-tracking provider, the same tracking page could show the live position of the vehicle on a map and an estimated time of arrival, and could send an automatic SMS or WhatsApp message at each stage.

The third enhancement is a mobile application for drivers, so that the driver can accept an assigned trip, upload the loading slip and the proof of delivery, and update the status from the road instead of telephoning the office. Together with secure payment collection and digital documentation, these enhancements would turn the present front-end demonstration into a complete transport management system suitable for daily commercial use, and would let the firm serve more customers with the same staff — helping everyone with the same reliability that the tagline promises: "*WE DELIVER SAFE.*"

BIBLIOGRAPHY / REFERENCES

1. Rajib Mall, *Fundamentals of Software Engineering*, 5th edition, PHI Learning, 2018.
2. Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, 8th edition, McGraw-Hill Education, 2015.
3. Elias M. Awad, *Systems Analysis and Design*, 2nd edition, Galgotia Publications.
4. Elmasri and Navathe, *Fundamentals of Database Systems*, 7th edition, Pearson, 2016.
5. Silberschatz, Korth and Sudarshan, *Database System Concepts*, 6th edition, McGraw-Hill, 2011.
6. Jon Duckett, *HTML and CSS: Design and Build Websites*, Wiley, 2011.
7. Jon Duckett, *JavaScript and JQuery: Interactive Front-End Web Development*, Wiley, 2014.
8. MDN Web Docs, *HTML: HyperText Markup Language* — Mozilla. https://developer.mozilla.org/en-US/docs/Web/HTML
9. MDN Web Docs, *CSS: Cascading Style Sheets* — Mozilla. https://developer.mozilla.org/en-US/docs/Web/CSS
10. MDN Web Docs, *Window.localStorage — Web Storage API* — Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
11. W3C, *HTML Living Standard*. https://html.spec.whatwg.org/
12. R. Fielding and J. Reschke (eds.), *Hypertext Transfer Protocol (HTTP/1.1): Message Syntax and Routing*, RFC 7230, IETF, 2014.
13. Google Fonts — Barlow Condensed and Inter. https://fonts.google.com/
14. Font Awesome Free 6.5.2 — icon library. https://fontawesome.com/
15. Tailwind CSS — utility-first CSS framework documentation. https://tailwindcss.com/docs
16. RP TRANSPORT COMPANY (RPTC), company profile, service list, fleet categories and contact details as published on the project website and in the project README.
17. Project source code and README, Git repository *rptc1* (RPTC transport management website), GitHub.